A Simpler Algorithm for Integer Multiplication Below $n \log n$, with a Formally Verified Proof

Piaoyang Cui
Google DeepMind

Yiming Liu
Google DeepMind

David P. Woodruff
Google Research & Carnegie Mellon University

Abstract

Schönhage and Strassen conjectured that multiplying two n-bit integers requires $\Omega(n \log n)$ time on multitape Turing machines. Harvey and van der Hoeven attained $O(n \log n)$, and a recent OpenAI preprint obtained $O\big(n(\log n)^{1-\kappa}\big)$ time using two fixed finite linear networks. We give a simpler construction around those networks, together with a machine-checked proof.

Short convolutions replace cross-axis Fourier phases by fractional shifts on one power-of-two grid. A small complex phase separates two frequency contributions that would otherwise tie, allowing stable recovery of the original transform. The FFT's own pairwise sums and differences also permute address-bit positions exactly, eliminating separate data-reordering passes. These changes allow a direct complex tensor FFT without Chinese remainder theorem (CRT) reductions, grid expansion, Bluestein transforms, or a polynomial-ring middle transform.

The written proof gives $\kappa = 2^{-56}$ at the preprint's unchanged tape-primitive constants, where it obtains 2^{-182}. An independent Lean 4 formalization proves the entire construction, including the networks and an inner multiplier, and gives $\kappa = 2^{-46}$ using only Lean's three standard logical axioms. Both proofs count tape movement and rounding error. The networks become useful only at astronomical input sizes, and we claim no practical speedup.

1 Introduction

1.1 Integer Multiplication on Sequential Tapes

How many steps are needed to multiply two n-bit integers? We use deterministic multitape Turing machines, with a fixed finite alphabet, fixed control, and a fixed number of one-dimensional tapes. Every tape-head movement counts.

A Fourier reduction splits the inputs into $\Theta(\log n)$-bit digits and computes a convolution of length $R = \Theta(n/\log n)$. Recovering sums of up to R digit products requires precision $P = \Theta(\log n)$, so the data occupy volume $V = \Theta(RP) = \Theta(n)$ bits. Although an FFT uses only $O(R \log R) = O(n)$ arithmetic operations, a tape head cannot jump to a distant entry. Scanning or rearranging this volume at each of the $\Theta(\log n)$ FFT stages costs $\Theta(n \log n)$ head steps, even if arithmetic is free. This is a limitation of that implementation strategy, not a general lower bound.

Schönhage and Strassen [18] achieved $O(n \log n \log\log n)$ time in 1971 and conjectured an $\Omega(n \log n)$ lower bound. Later work reduced the overhead [6, 7, 3, 10], until Harvey and van der Hoeven [8] reached $O(n \log n)$. The recent integer-multiplication and exact-DFT preprints [16, 17] obtain a power saving below $n \log n$; see Table 1.

Reference	Running time	Main technique
Schönhage–Strassen [18]	$O(n \log n \log\log n)$	Recursive Fermat-ring FFTs
Fürer [6, 7]; De et al. [3]	$n \log n \, 2^{O(\log^* n)}$	Synthetic roots in polynomial rings
Harvey–van der Hoeven–Lecerf [10]	$O(n \log n \, 8^{\log^* n})$	Sharpened synthetic-root construction
Harvey–van der Hoeven [8]	$O(n \log n)$	Multi-prime CRT and Gaussian resampling
OpenAI [16]	$O\big(n(\log n)^{1-\kappa}\big)$	Fixed networks; multi-grid Bluestein construction
This paper	$O\big(n(\log n)^{1-\kappa}\big)$	Fixed networks on one power-of-two grid

Table 1: Deterministic multitape multiplication. Here $\kappa > 0$ is an explicit constant, and $\log^* n$ is the iterated logarithm.

The saving in [16] comes from two fixed finite linear networks that batch exchanges of address fields (blocks of address bits) and Fourier butterflies (pairwise sums and differences). We ask whether these networks can support multiplication on one power-of-two grid, with a complete machine-checked proof.

1.2 Our Results

We keep one grid by representing fractional shifts with short convolutions and permuting address-bit positions by exact butterfly cancellations. This gives a direct complex tensor FFT, which applies a Fourier transform along each axis. The written proof imports exactly the tape primitives of [16, Lemmas 3–4, Lemma 12, and Proposition 14] at their printed constants (Facts 3.3, 3.5 and 3.6). We prove the remaining analytic and algorithmic steps, including precision and tape-layout bounds.

Theorem 1.1 (Single-grid integer multiplication)**.** *There is a deterministic multitape Turing machine with a fixed finite alphabet and a fixed finite number of tapes that multiplies two n-bit integers in worst-case time*

$$O\big(n(\log n)^{1-\kappa}\big), \qquad \kappa = 2^{-56}.$$

The machine in Algorithm 1 uses one grid $R = t_1 \cdots t_d = 2^k$, fractional-shift kernels with two-term cyclic solvers, and a direct complex tensor FFT. Above a fixed cutoff n_*, all delegated integer

products have $O((\log n)^2)$-bit operands. These products and inputs below n_* use any fixed independent deterministic multiplier of cost $\mathsf{M}(N) = N(\log N)^{O(1)}$, such as [18]; there is no self-recursion.

Theorem 1.2 (Formally verified bound)**.** *The Lean 4 formalization [4, 12, 11] proves the entire construction from scratch in Mathlib's finite-alphabet multi-stack Turing machine model over $\mathbb{N}$, certifying worst-case time $O(n(\log n)^{1-\kappa})$ with $\kappa = 2^{-46}$. It proves both $h = 100$ networks, all stream, field-interchange, and butterfly primitives, and the sharpened leaf-cutoff and guard bounds*

$$(\beta, C_1, 1-\tau, 1-\sigma) = (1/64, 14, 2^{-40}, 2^{-42}).$$

Here β controls when network recursion stops, C_1 is the degree of the temporary-precision bound, and τ, σ are the interchange and butterfly exponents. It also proves an independent two-dimensional $O(N \log N)$ inner multiplier [13]. The proof uses only Lean's three standard logical axioms, `propext`, `Classical.choice`, and `Quot.sound`. It assumes no result from [16, 8].

How to read the results. Section 7 proves the written bound, charging every tape movement and rounding error. Its recursive tape routines and independent short products never invoke the new multiplier itself. The Lean proof also checks the primitives; raising its time bound to the power 2^{46} allows a specification in natural-number arithmetic. Section 8 explains the finite-alphabet clause (no unbounded integers in one cell), the tape simulation, and the trusted kernel and axioms. The full paper [5] gives the complete analysis in greater detail; the companion paper [13] develops the independent two-dimensional $O(N \log N)$ multiplier that serves as the inner multiplier in the formalization.

Both bounds are galactic: the fixed $h = 100$, $m = 10^6$ networks become useful only at astronomical input sizes. Our exact-arithmetic Python package [2] tests the multi-axis pipeline with small substitute networks and checks the $h = 100$ certificates symbolically, rather than executing those networks at their required sizes. We claim neither a practical speedup nor a practical implementation of the large-network algorithm.

2 Overview

2.1 What the Fixed Networks Provide

Strassen's 2×2 identity replaces eight block multiplications by seven and linear combinations. Repeating it changes the exponent from 3 to $\log_2 7$. The tape networks use the same principle: a fixed reduction in the number of smaller calls changes an exponent.

A *role* is one of W equal-length data streams on separate tapes. A width-m task groups m elementary operations per role, while a width-1 call performs one elementary operation. The direct implementation therefore takes Wm calls, whereas a network uses $s < Wm$ calls and $O(V)$ linear-combination work on V data bits. Dividing by the number of roles gives the *normalized count* s/W. If $s/W < m^\gamma$ for some $\gamma < 1$, then t levels handle width $u = m^t$ with normalized count

$$(s/W)^t \le (m^t)^\gamma = u^\gamma,$$

rather than u; the constants W, m, s remain fixed.

The two identities of [16], with $h = 100$ and $m = h^3 = 10^6$, give exponents $\tau, \sigma < 1$ for field interchange and butterfly recursion, respectively. The printed interchange routine exchanges two disjoint u-bit address fields in $O(V(1 + u^\tau))$ time; the butterfly routine accepts the following layout.

We store the length-R vector on one tape as a balanced d-axis array: for fixed $\varepsilon, c > 0$, choose

$$R = 2^k = t_1 \cdots t_d, \qquad d = \lfloor k^\varepsilon \rfloor, \quad \ell = \lceil k/d \rceil, \quad t_d = r = 2^\ell, \quad t_a \in \{r/2, r\}.$$

We work above a fixed cutoff with $d \geq 2$. Axis d varies fastest, so its r contiguous complex entries form one *record*, and there are $M = R/r = 2^{m_0}$ records. An m_0-bit counter supplies the record address, which is not stored with the data. At working precision P, we use $q = P + 2$ fractional bits and width $w = q + 2$ per real or imaginary component, giving volume $V = 2Mrw = \Theta(RP)$.

A *chunk* consists of $K = \lfloor d^c \rfloor$ consecutive record-address bits. A *slice* selects the same offset in each of $D \leq d$ consecutive chunks. For illustration, offset 1 from the least significant end of $(x_5x_4x_3 \mid x_2x_1x_0)$ selects x_4, x_1.

A butterfly on one address bit pairs records differing only at that bit and applies $H_0(u, v) = ((u + v)/2, (u - v)/2)$ to corresponding entries. One parallel call applies the tensor product of these maps on the entire slice, rounding only its final output. It cannot skip individual slice bits: the interface specifies whole chunks and a common offset, not an arbitrary subset. Its cost is $O(Vd^{\lambda'})$ for a fixed $\lambda' < 1$, including the network's implementation on fixed tapes (Fact 3.6).

If the current bit of every outer axis occupies a slice, one call performs a Fourier round across those axes. There are ℓ rounds, and our parameters give $V = \Theta(n)$ and $k = \Theta(\log n)$. Since $\ell = \Theta(k/d)$, their cost is

$$O(V\ell d^{\lambda'}) = O(Vk\, d^{\lambda'-1}) = O\big(n(\log n)^{1-\varepsilon(1-\lambda')}\big).$$

We must connect multiplication to these rounds without spending $O(n \log n)$ elsewhere.

2.2 The Multiplication Pipeline

Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^R$ be digit vectors padded to at least twice their original length, so that cyclic convolution, denoted $\circledast$, equals ordinary convolution. Packing them into $z = \mathbf{a} + i\mathbf{b}$ gives

$$z \circledast z = \mathbf{a} \circledast \mathbf{a} - \mathbf{b} \circledast \mathbf{b} + 2i(\mathbf{a} \circledast \mathbf{b}),$$

so its imaginary part isolates twice the desired product.

We use the normalized forward transform $(\mathcal{F}_R w)_u = R^{-1} \sum_{j<R} w_j e^{2\pi i ju/R}$. Conjugation reverses the Fourier sign, giving $\mathcal{F}_R(\overline{\mathcal{F}_R w}) = R^{-1}\overline{w}$. The convolution theorem gives $\mathcal{F}_R(z \circledast z) = R(\mathcal{F}_R z) \odot (\mathcal{F}_R z)$, where $\odot$ is entrywise multiplication. Thus a second forward call on the conjugated square returns $\overline{z \circledast z}/R^2$, and

$$\mathbf{a} \circledast \mathbf{b} = -\frac{R^2}{2} \operatorname{Im} \mathcal{F}_R\Big(\overline{(\mathcal{F}_R z) \odot (\mathcal{F}_R z)}\Big).$$

The conjugate lets us reuse the forward routine instead of building a separate inverse.

Each forward call is implemented by Bridge, the three middle stages in Figure 1. Spreading represents fractional sample positions on the integer grid, TensorFFT transforms along each axis, and deconvolution recovers the one-dimensional Fourier coefficients. Write $\widetilde{\mathbb{C}}_q = 2^{-q}(\mathbb{Z} + i\mathbb{Z})$ for the fixed-point grid and $\mathcal{Q}_q$ for coordinatewise nearest rounding, with ties to even. The precision in Algorithm 1 leaves coefficient error below $1/2$, so integer rounding and carry propagation recover the exact product (Proposition 7.2).

Two obstacles. A tensor DFT omits the cross-axis phases of the one-dimensional DFT. The CRT construction of [8, 16] separates the transform using coprime odd-prime axis lengths, and Gaussian resampling then interpolates onto larger power-of-two grids. We instead account for the missing phases by fractional shifts on the existing grid.

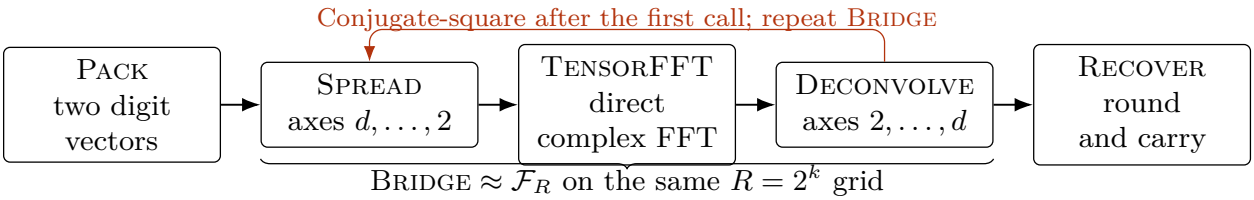


Figure 1: The three middle boxes form one forward call. Follow the return arrow once, conjugate-squaring between calls; recovery follows the second call.

Algorithm 1 Integer multiplication on one power-of-two grid

Input: $0 \le A, B < 2^n$. **Output:** AB.

Parameters for $n \ge n_*$**:** $b = \lfloor \log_2 n \rfloor$, $N_{\mathrm{dig}} = \lceil n/b \rceil$, least power of two $R = 2^k \ge 2N_{\mathrm{dig}}$, and balanced axes. Set $p = 2b + 2k + 6$; use the precision schedule of Theorem 4.6 and the parameter choices of Lemma 7.1.

1: If $n < n_*$, return the product from the independent multiplier M.

2: $x \leftarrow \textsc{Pack}(A, B)$: set $x_j = 2^{-b}(a_j + ib_j)$ from radix-2^b digits, with zero padding.

3: $\hat{y} \leftarrow \textsc{Bridge}(x)$.

4: $\hat{w} \leftarrow \mathcal{Q}_{p+2}\big(\overline{\hat{y}} \odot \hat{y}\big)$.

5: $\hat{z} \leftarrow \textsc{Bridge}(\hat{w})$.

6: Return $\textsc{Recover}(\hat{z})$: round $-2^{2b-1}R^2 \operatorname{Im} \hat{z}_j$ to integers and propagate carries.

Subroutine $\textsc{Bridge}(x)$**.** Use the power-of-two scale S and fixed-point operations of Algorithm 2. The kernels g_a, offsets δ_a, and mixing weights c_0, c_1 are described below; put $A_a = \prod_{i<a} t_i$.

7: Set $U \leftarrow x/S$ exactly and reverse the input axis fields to tensor order. For $a = d, \dots, 2$, spread along axis a using g_a at offsets δ_a.

8: $Y \leftarrow \textsc{TensorFFT}(U)$.

9: For $a = 2, \dots, d$, solve $c_0 Z_f + c_1 Z_{(f - R/A_a) \bmod R} = Y_f$ for $0 \le f < R$ by the rounded two-pass cyclic solver; set $Y \leftarrow Z$.

10: Return $\mathcal{Q}_{p+2}(SY)$.

The parallel primitive also requires aligned slices and supplies only the real butterfly H_0. In [16], physical chunk swaps move address blocks into place, and Bluestein's chirp identity [1] replaces Fourier phases by convolution with quadratic-phase sequences. Polynomial transforms [14, 15] compute these convolutions over $\mathbb{C}[y]/(y^r + 1)$, using coefficient shifts to implement roots of unity. We instead use exact address-bit permutations and a direct complex tensor FFT (Table 2).

Prior construction [8, 16]	**Single-grid construction**	**Analysis**
CRT and Gaussian grid expansion	Fractional shifts on the unchanged 2^k grid	Section 4
$O(d^2)$ adjacent axis swaps	$O(d)$ direct swaps with the innermost axis	Lemma 4.4
Physical width-K chunk swaps	Bit-exact address permutations from butterflies	Theorem 5.6
Bluestein chirps and polynomial-ring convolutions	Direct complex tensor FFT	Theorem 6.4
Middle error amplified by M; output widening	Error $< 12\ell 2^{-q}$; no output widening	Corollary 6.5
$\Theta(rP)$-bit packed polynomial products	$O(P^2)$-bit delegated products	Section 7.1

Table 2: Changes around the shared networks. Axis swaps remain in spreading; separate chunk-swap layout passes inside the middle transform are removed.

2.3 Idea 1: Fractional Shifts on One Grid

The missing cross-axis phases can be represented as fractional sample positions. For two axes, write $R = t_1 r$, input position $j = j_1 + t_1 j_2$, and output frequency $f = u_1 r + u_2$. These use opposite axis orders; the initial field reversal in Algorithm 1 converts the input to tensor order. Expanding the Fourier phase gives

$$\frac{jf}{R} = \frac{j_1 u_1}{t_1} + \left(j_2 + \frac{j_1}{t_1}\right)\frac{u_2}{r} + j_2 u_1.$$

The integer term $j_2 u_1$ disappears from the exponential, leaving axis-2 samples at positions $j_2 + \delta$, where $\delta = j_1/t_1$. For illustration, with $t_1 = 2, r = 4$, even-indexed input samples lie at integer positions and odd-indexed samples at half-integer positions. We must represent this half-cell displacement on the integer grid.

We do so by a short convolution. The ideal interpolation kernel $\mathrm{sinc}(s) = \sin(\pi s)/(\pi s)$, with $\mathrm{sinc}(0) = 1$, decays too slowly, so we damp it by a Gaussian:

$$g_0(s) = \mathrm{sinc}(s) e^{-\pi s^2/\alpha^2}, \qquad G(\xi) = \int_{\xi-1/2}^{\xi+1/2} \alpha e^{-\pi \alpha^2 v^2}\, dv.$$

Here G is the Fourier transform of g_0 (with kernel $e^{-2\pi i \xi s}$), and Definition 4.1 chooses the width α from a parameter ω that controls the exponentially small tails. For a kernel g, a sample at $j_2 + \delta$ contributes $g(j_2 - h + \delta)$ at integer position h. We retain only the $2P + 1$ integer offsets $-P, \ldots, P$, called *taps*, and then fold indices modulo r; no grid points are added (Lemma 4.5 and Theorem 4.6).

Sampling a kernel on integers creates shifted frequency copies, called *aliases*. Before truncation and scaling, Poisson summation gives

$$(\mathcal{F}_{t_1,r} U)_{u_1,u_2} = \sum_{\nu \in \mathbb{Z}} \widehat{g}\left(\frac{u_2}{r} - \nu\right) (\mathcal{F}_R x)_{((u_1-\nu) \bmod t_1) r + u_2}.$$

For fixed u_2, write Y_{u_1} for this output and $Z_{u_1} = (\mathcal{F}_R x)_{u_1 r + u_2}$. With $g = g_0$, the weights $c_0 = G(u_2/r)$ and $c_1 = G(u_2/r - 1)$ dominate: all other aliases have total weight $O(2^{-\omega})$. Recovery therefore requires undoing the cyclic mixture $Y_{u_1} \approx c_0 Z_{u_1} + c_1 Z_{(u_1-1) \bmod t_1}$.

Separating the two aliases. The real kernel fails at the midpoint frequency, called the Nyquist bin. At $u_2 = r/2$, evenness gives $c_0 = G(1/2) = G(-1/2) = c_1$. The retained mixing map $c_0(I + T)$, where T shifts once around the cycle, annihilates $(1, -1, 1, -1, \ldots)$ on even cycles.

Every real kernel has the same magnitude obstruction: Hermitian symmetry, $\widehat{g}(-\xi) = \overline{\widehat{g}(\xi)}$, ties the alias magnitudes at $1/2$. If $c_1 \neq 0$, some Lth root of unity ζ lies within π/L of $-c_0/c_1$, so $|c_0 + c_1 \zeta| \leq \pi |c_1|/L$. Its Fourier eigenvector forces inverse norm at least $L/(\pi|c_1|)$ whenever the inverse exists. Since the cycle at axis a has length $A_a = \prod_{i<a} t_i$, paying this factor at every axis (for bounded weights) costs $\sum_{a=2}^{d} \log_2 A_a = \Theta(dk)$ bits on a balanced grid, instead of $O(k)$.

We break the symmetry by multiplying the kernel by a small complex phase, called a *carrier*. The negative half-bin sign gives

$$g_a(s) = e^{-\pi i s/t_a} g_0(s), \qquad \widehat{g}_a(\xi) = G\left(\xi + \frac{1}{2t_a}\right), \qquad y = \frac{u_a + 1/2}{t_a}.$$

Now $c_0 = G(y)$ and $c_1 = G(y - 1)$. Since t_a is even, every sampled y lies inside $(0, 1)$ and at distance at least $1/(2t_a)$ from $1/2$ (Figure 2). For $B_a = c_0 I + c_1 T$, Lemma 4.3 shows $|c_0 - c_1| \geq \frac{7}{4t_a}$, so the amplification of maximum-entry error is $\|B_a^{-1}\|_\infty \leq 1/|c_0 - c_1| \leq \frac{4}{7} t_a$, independently of the cycle length.

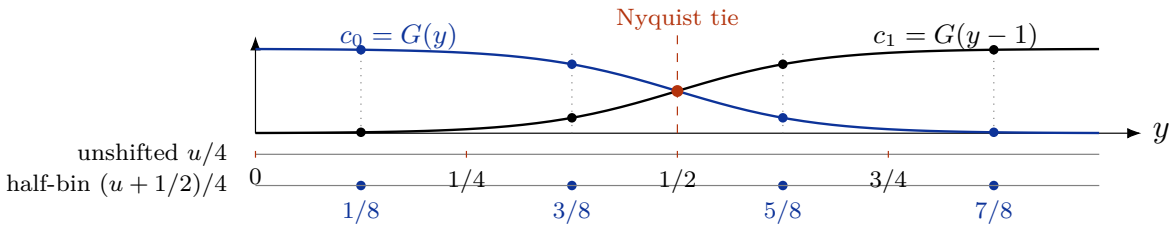


Figure 2: Schematic alias weights for $t_a = 4$. The half-bin samples avoid the equal-weight crossing and stay inside $(0, 1)$, unlike the unshifted samples.

The sign also matters for the omitted aliases. A positive carrier puts $u_a = 0$ at $y = -1/(2t_a)$, outside $(0, 1)$, where the discarded $\nu = -1$ alias need not satisfy the uniform $O(2^{-\omega})$ tail bound. The negative sign protects both the weight gap and the two-alias approximation (Section 4.2).

Axis order. For several axes, the fractional shift on axis a depends only on earlier input digits:

$$A_a = \prod_{i<a} t_i, \qquad \delta_a(\mathbf{j}) = \frac{\sum_{i<a} j_i A_i}{A_a}, \qquad f(\mathbf{u}) = \sum_{a=1}^{d} u_a \frac{R}{A_{a+1}}.$$

We spread in descending order $d, \ldots, 2$, so those earlier digits remain unchanged until axis a reads them. The initial field reversal makes axis d contiguous, while tensor frequency order agrees with increasing f.

Let $\mathcal{A}_a$ mix all aliases of axis a, and put $\mathbf{t} = (t_1, \ldots, t_d)$. For the exact, unscaled spread array, Lemma 4.2 gives $\mathcal{F}_{\mathbf{t}} U = \mathcal{A}_2 \mathcal{A}_3 \cdots \mathcal{A}_d (\mathcal{F}_R x)$, with the rightmost operator acting first. Subtracting R/A_a changes only earlier frequency digits, which makes this ordered factorization work. We remove the retained two-term factors from the outside in, starting with B_2 and proceeding in ascending order; they need not commute. Their inverse norms multiply to at most $\prod_{a=2}^{d} (4t_a/7) = (4/7)^{d-1} R/t_1 \le R/t_1$, so the total loss needs only $O(k)$ extra precision bits (Lemma 4.3).

Two-pass cyclic solves. Divide by the larger weight and orient the cycle to obtain $Z_j = \widetilde{Y}_j - \rho Z_{j-1 \bmod L}$, where $\rho = \min(c_0, c_1)/\max(c_0, c_1) < 1$. For illustration, on a length-4 cycle a pass starting from seed zero ends at

$$\widetilde{Y}_3 - \rho \widetilde{Y}_2 + \rho^2 \widetilde{Y}_1 - \rho^3 \widetilde{Y}_0 = (1 - \rho^4) Z_3.$$

Dividing by $1 - \rho^4$ recovers Z_3, and a second pass from that seed recovers Z_0, Z_1, Z_2.

In general, $L = A_a$ is an even power of two, so a trip around the cycle multiplies the unknown seed by $(-\rho)^L = \rho^L$. The zero-seed endpoint is therefore $(1 - \rho^L) Z_{L-1}$, and the same two-pass procedure applies. Per-step recurrence errors accumulate geometrically, with amplification

$$\frac{1 + \rho + \cdots + \rho^{L-1}}{1 - \rho^L} = \frac{1}{1 - \rho}.$$

Even when ρ is close to one, this factor depends on the weight gap, not on L; Lemma 4.5 also accounts for stored-coefficient errors and the closing division.

The cyclic solves need no transpose: writing $f = j(R/L) + s$, for each fixed column s the successive rows j form one cycle. A side tape holds one $O(P)$-bit state per column, and each row sweep advances all states in lockstep. Each pass has $O(V)$ scan overhead (Lemma 4.5).

Spreading does need each axis to be contiguous. We swap it directly with the innermost axis and then restore it, using $O(d)$ field swaps rather than $O(d^2)$ adjacent swaps. Lemmas 4.4 and 4.5 give the tape costs and implement each short convolution using $O(P^2)$-bit integer products.

Scaling and rounding. The butterfly primitive requires inputs in the closed unit disk. For the exact folded kernel $H_\delta^{(a)}$ of Definition 4.1, Poisson summation expresses each unnormalized Fourier coefficient as phases with nonnegative weights $G(y-\nu)$. These weights sum to one because their defining integration intervals tile $\mathbb{R}$. Parseval and Cauchy–Schwarz then give $\left\|H_\delta^{(a)}\right\|_2 \le 1$ and $\left\|H_\delta^{(a)}\right\|_1 \le \sqrt{t_a}$, so spreading amplifies the sup norm by at most $\sqrt{R/t_1}$.

We therefore divide the input by a power of two S that exceeds $X\sqrt{R/t_1}$, where $X = \max(1, \|x\|_\infty)$, by a fixed safety factor; Algorithm 2 and Theorem 4.6 choose S so that the division is exact and every computed spread, including its rounding errors, stays in the disk.

Two rounding rules serve different purposes. Coordinatewise truncation toward zero, Q_q, cannot increase either coordinate's magnitude, so butterflies, permutations, and disk products use it to preserve $\mathbb{D}_q = \{z \in \widetilde{\mathbb{C}}_q : |z| \le 1\}$. Nearest rounding $\mathcal{Q}_q$ halves the worst-case coordinate error but can leave the disk: at mesh $1/2$, $(3+4i)/5$ rounds to $1/2+i$. We use nearest rounding in spreading and deconvolution, where separate modulus bounds allow this increase.

For $p \ge \max(8,k)$, working precision $P = O(p+k)$ suffices even as d grows. For $x \in \widetilde{\mathbb{C}}_{p+2}^R$ with $\|x\|_\infty \le 2^p$, Theorem 4.6 gives maximum-entry error $\|\textsc{Bridge}(x) - \mathcal{F}_R x\|_\infty < 2^{-p}X/4$, including tap truncation, omitted aliases, cyclic solves, and rescaling. That theorem specifies ω, P, q, w and all safety margins. Taps, solver scalars, and phases use certified rational power-series approximations, not a transcendental rounding oracle (Lemma A.1).

2.4 Idea 2: Exact Permutations of Address Bits

The tensor FFT needs to rearrange record-address bits to align its parallel rounds. We do this by temporarily mixing records and then canceling the mixing exactly. For a two-bit illustration, a controlled flip sends (x_c, x_t) to $(x_c, x_t \oplus x_c)$, where $\oplus$ is addition modulo two. It changes the target bit only when the control is one. The identities

$$2H_0^2 = I, \qquad 2H_0\,\mathrm{diag}(1,-1)H_0 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

give these two cases: two target-bit butterflies turn the recordwise sign $(-1)^{x_c x_t}$ into a controlled flip of the address. The sign needs no routing, since an address counter determines whether to negate each record (Lemma 3.4).

Selecting part of a slice. A parallel call acts on every bit of its slice, not just the desired targets. We cancel its action on the other bits using $S_{\mathrm{ph}} = \mathrm{diag}(1, i)$ and the identities

$$H_0^3 = \tfrac{1}{2}H_0, \qquad (H_0 S_{\mathrm{ph}})^3 = \tfrac{1+i}{4}I.$$

Let T contain the s selected bits and U the other $D-s$ bits. Before each of three full layers, apply the recordwise phase $i^{\sum_{u\in U} x_u}$. Each selected bit leaves a factor $1/2$, and each unselected bit leaves the scalar $(1+i)/4$. Multiplying afterward by $2^s[2(1-i)]^{D-s}$ therefore gives exactly H_T, the butterfly on T alone.

For disjoint control-target pairs, the controlled sign $(\mathrm{CZ}z)(x) = (-1)^{\sum_j x_{c_j} x_{t_j}} z(x)$ is also one scan. A batch of controlled flips is $\mathrm{CNOT}^{\otimes s} = 2^s H_T \mathrm{CZ} H_T$, so it uses six full layers (Figure 3). Unselected bits are restored before CZ reads them, so they may serve as controls; controls may also lie far from the target slice.

Every permutation of address-bit positions factors into two involutions, each consisting of disjoint swaps and fixed positions. This follows by expressing each cycle as two reflections (Lemma 5.5).

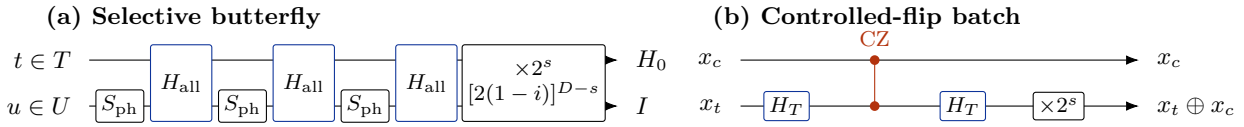


Figure 3: Classical linear circuits, read left to right. In (a), three full slice layers H_{all} cancel on U; in (b), two selective maps turn a controlled sign into controlled flips.

A swap uses three XOR updates: $(a, b) \mapsto (a \oplus b, b) \mapsto (a \oplus b, a) \mapsto (b, a)$. The two involutions therefore give six stages of disjoint controlled flips. Grouping targets by slice, with leftover bits handled by elementary stream operations, gives total cost $O(V\ell d^{\lambda'})$ (Theorem 5.6).

Bit-exact implementation. Before each controlled-flip batch, append $8d$ zero fractional bits, giving precision $q_* = q + 8d$. Six layers on $D \le d$ bits divide by at most 2^{6D}. All phases and restoring scalars are Gaussian integers, with integer real and imaginary parts, so every exact layer-boundary value is representable on the lifted grid. The imported primitive evaluates each layer without internal rounding and then applies Q_{q_*} once; this truncation discards only zeros.

Every layer call also receives a disk-valued array: normalized layers and phases preserve the disk, and each restoring scalar is applied in steps that increase moduli only toward the completed value, which is a contraction or a permutation. A completed batch merely permutes the original data, so removing the temporary bits is exact (Lemma 5.3 and Proposition 5.4).

We permute the $\Theta(\log n)$ address-bit positions, not arbitrary permutations of the M records. This includes a square power-of-two matrix transpose of these long records, since it exchanges two address fields. The method uses long payloads and complex linear combinations that later cancel, so it gives no short-record routing bound in the transposition comparison of [9].

2.5 Idea 3: A Direct Tensor FFT

Decimation in frequency (DIF) separates even and odd output frequencies. On a length-$2h$ vector, one step forms $(z_j + z_{j+h})/2$ and $(z_j - z_{j+h})e^{2\pi ij/(2h)}/2$ for $0 \le j < h$. The first half feeds the even frequencies and the second feeds the odd frequencies of two length-h transforms. Recursing therefore extracts frequency bits in reverse order.

We first compute the length-r FFT within each record, including local bit reversal. For the outer axes, an exact layout $\mathcal{L}$ places the at most $(d-1)K$ bits that do not fill complete chunks at the front and interleaves the remaining chunks. At most K boundary levels use sequential exact butterflies followed by one truncation per level, costing $O(VdK)$ in total. Every other level occupies one aligned slice and takes one parallel-layer call (Lemma 6.3 and Algorithm 4).

The complex phases for one outer round can be combined before rounding. Since every axis length divides r, write each active phase as $e^{2\pi iJ_a/r}$ for an integer J_a. Their product is the single table entry $\theta_{2J} = e^{2\pi iJ/r}$, where $J = \sum_{a<d} J_a \bmod r$ and $\theta_j = e^{\pi ij/r}$. An address counter computes J exactly, and one phase then multiplies every entry of that record.

Let B reverse bits within each outer frequency axis. The outer DIF rounds in layout $\mathcal{L}$ return the output in layout $\mathcal{L}B$, so one exact permutation $\mathcal{U} = (\mathcal{L}B)^{-1} = B^{-1}\mathcal{L}^{-1}$ restores natural order (Lemma 6.3). The order matters, since $\mathcal{L}$ moves bits between axis fields.

Stored phases lie in $\mathbb{D}_q$, and each complex product is formed exactly before one truncation. Combining phases before lookup gives error less than $6 \cdot 2^{-q}$ per level, independent of d; separately rounded phases would pay a factor of d. Exact rounds are contractions, so local errors add over ℓ inner levels and at most ℓ outer levels, while the permutations contribute zero error. Thus the tensor transform has error $< 12\ell\, 2^{-q}$ and needs no extra output precision (Theorem 6.4 and Corollary 6.5).

2.6 Putting It Together

The networks fix τ, σ and the guard degree C_1: their exact intermediate calculations need $O(d^{C_1})$ extra bits before rounding. Choosing $\varepsilon C_1 < 1$ makes this $o(P)$ and keeps the guarded tape volume $\Theta(n)$. Within butterfly recursion, the leaf-cutoff exponent β balances depth against the number of roles, while c controls chunk width. These parameters determine the allowed λ' (Fact 3.6).

Write $\mathsf{m}(N) = \mathsf{M}(N)/N = (\lg(2N))^{O(1)}$ for the inner multiplier's cost per operand bit. The size choices of Lemma 7.1 give $P = \Theta(\log n)$, $V = \Theta(n)$, $d = \Theta(P^{\varepsilon})$, $\ell = \Theta(P^{1-\varepsilon})$, and $K = \Theta(P^{\varepsilon c})$. In the table, a gap g means cost per bit of volume $O(P^{1-g})$, up to fixed powers of $\log P$.

Work	**Cost divided by V**	**Exponent gap**
Boundary butterfly levels	dK	$1 - \varepsilon(1 + c)$
Parallel layers and exact layouts	$\ell d^{\lambda'}$	$\varepsilon(1 - \lambda')$
Record FFTs and phases	$\ell\, \mathsf{m}(P)$	ε
Direct axis swaps	$d(1 + \ell^{\tau})$	$(1 - \varepsilon)(1 - \tau)$
Spreading and deconvolution	$d(\mathsf{m}(P) + \mathsf{m}(P^2))$	$1 - \varepsilon$

The first row pays for at most K boundary levels, each using d ordinary butterfly passes. The second pays for ℓ parallel rounds and the exact layout permutations. The third covers ℓ levels of P-bit arithmetic for record FFTs and phase products. Exposing and restoring axes uses $O(d)$ field swaps of width at most ℓ, giving the fourth row. Finally, spreading and cyclic solves use $O(P^2)$-bit packed products and $O(P)$-bit scalars, giving the fifth row.

Packing, conjugated squaring, carry recovery, and table generation fit within these bounds (Section 7.1). Any κ strictly below all five gaps absorbs the remaining polylogarithmic factors and gives total time $O(n(\log n)^{1-\kappa})$.

The cost of physical chunk swaps. Using $O(P/K)$ physical width-K swaps for the layout would add normalized cost $PK^{\tau-1}$, with gap $\varepsilon c(1 - \tau)$. The butterfly implementation uses an intermediate recursion exponent $\lambda < 1$ and requires $\tau(1 + c/\beta) < \lambda$, hence $c < \beta(1 - \tau)/\tau$. Together with $\varepsilon < 1/C_1$, this would force

$$\kappa < \varepsilon c(1 - \tau) < \frac{\beta(1 - \tau)^2}{C_1 \tau}.$$

The deficit $1 - \tau$ enters twice: through the speed of each swap and through the permitted chunk width. Exact permutations remove this cost, so the limiting gap is linear in the deficit.

The resulting exponent. The rational parameter choice in Section 7.1 uses the unchanged printed primitive constants of [16]. It makes all five gaps positive, with minimum

$$g_{\min} = g_{\mathrm{lay}} = \varepsilon(1 - \lambda') = 3 \cdot 2^{-57} > 2^{-56}.$$

The strict margin absorbs the logarithmic factors and proves Theorem 1.1; Equation (11) gives the full comparison. At those same constants, [16] obtains $\kappa = 2^{-182}$, so the written improvement comes from the architecture, not stronger network bounds. Separately, the sharpened leaf-cutoff and guard bounds certified in Lean give $\kappa = 2^{-46}$ (Remark 7.3); they are not used in the written proof.

Proof roadmap. Section 3 fixes the model and states the three imported primitives, and Section 4 proves the bridge. Sections 5 and 6 construct the exact permutations and the tensor FFT, Section 7 proves Theorem 1.1, Section 8 covers the formal proof, and Section A the scalar tables.

3 Preliminaries

We fix tape and rounding conventions and derive the two-call convolution identity. The three primitives below are our only imports from [16].

3.1 Multitape Model, Tensor Layout, and Fixed-Point Grids

Write $[N] = \{0, \dots, N-1\}$ and $\lg N = \max\{1, \lceil \log_2 N \rceil\}$. For complex vectors and matrices, use $\|x\|_\infty = \max_j |x_j|$ and $\|A\|_\infty = \max_i \sum_j |A_{ij}|$. We also write $\|x\|_1 = \sum_j |x_j|$ and $\|x\|_2^2 = \sum_j |x_j|^2$. Our deterministic machines have fixed finite alphabets and a fixed number of one-dimensional tapes. Scans, rewinds, counters, and cleanup count. Arrays are row-major: the last coordinate varies fastest, and position determines the address. An array of M *records*, contiguous blocks of r complex entries at w bits per component, has volume $V = 2Mrw$.

A *spectator* is an address field left unchanged by an operation. Parameters are stored in headers using the prefix-free code $\Gamma(N) = 0^{b_N} \mathrm{bin}_{b_N}(N)$ for $N > 0$, where $b_N = \lfloor \log_2 N \rfloor + 1$. Here $\mathrm{bin}_b(N)$ is the b-bit binary expansion, so the code has length $2b_N$.

Definition 3.1 (Grids and rounding)**.** Set $\widetilde{\mathbb{C}}_P = 2^{-P}(\mathbb{Z} + i\mathbb{Z})$ and let $\mathbb{D}_P = \{z \in \widetilde{\mathbb{C}}_P : |z| \le 1\}$ be its unit disk. A disk record lies in $\mathbb{D}_q^r$. Components are signed two's-complement integers at scale 2^{-q}, most significant bit first, with the real component first. Width $w \ge q + 2$ represents both endpoints ± 1. Entrywise, $\mathcal{Q}_P$ rounds coordinates to nearest grid points, with ties to even, and Q_P truncates coordinates toward zero.

For a complex scalar z,

$$|\mathcal{Q}_P z - z| \le \frac{2^{-P}}{\sqrt{2}}, \qquad |Q_q z| \le |z|, \qquad |Q_q z - z| < \sqrt{2}\, 2^{-q}. \tag{1}$$

Truncation preserves the disk required by Fact 3.6. Nearest rounding halves the error bound but can leave the disk. Scalar products use a fixed independent integer multiplier $\mathsf{M}(N) = N\mathsf{m}(N)$, where $\mathsf{m}(N) = (\lg(2N))^{O(1)}$, for example [18]. It never calls the multiplier constructed here.

3.2 Normalized Transforms and Two-Call Convolution

Definition 3.2 (Normalized transforms)**.** For $\mathbf{t} = (t_1, \dots, t_d)$ and $R = \prod_a t_a$, define

$$(\mathcal{F}_R x)_u = \frac{1}{R} \sum_{j<R} x_j e^{2\pi i j u/R}, \qquad (\mathcal{F}_{\mathbf{t}} U)_{\mathbf{u}} = \frac{1}{R} \sum_{\mathbf{j}} U_{\mathbf{j}} e^{2\pi i \sum_a j_a u_a / t_a}.$$

Write $(a \circledast b)_j = \sum_{\ell<R} a_\ell b_{(j-\ell) \bmod R}$ for cyclic convolution and $\odot$ for entrywise multiplication.

Both transforms have infinity norm at most one, since each row has absolute sum one. The convolution theorem gives $\mathcal{F}_R(a \circledast b) = R(\mathcal{F}_R a) \odot (\mathcal{F}_R b)$. For real a, b, set $z = a + ib$ and $w = z \circledast z$. Then $\operatorname{Im} w = 2(a \circledast b)$ and $(\mathcal{F}_R z) \odot (\mathcal{F}_R z) = R^{-1}\mathcal{F}_R w$. Conjugation reverses the Fourier sign, so orthogonality gives $\mathcal{F}_R(\overline{\mathcal{F}_R w}) = \overline{w}/R$: the inner sum $\sum_{u<R} e^{2\pi i(j-\ell)u/R}$ is R for $j = \ell$ and zero otherwise. Hence two forward calls suffice [13]:

$$a \circledast b = -\frac{R^2}{2} \operatorname{Im}\Big(\mathcal{F}_R\big(\overline{(\mathcal{F}_R z) \odot (\mathcal{F}_R z)}\big)\Big). \tag{2}$$

3.3 Imported Stream and Network Primitives

A *stream* is a list of records read sequentially.

Fact 3.3 (Stream operations [16, Lemmas 3–4])**.** *At fixed radix, the following cost $O(V)$ on a fixed number of tapes, including preparation, counters, rewinds, and cleanup: a fixed permutation of two address digits; moving one address digit; splitting on one digit, applying a fixed pointwise stream operation, and merging; and distributing whole rows among a fixed number W of streams by row index modulo W, then merging in any fixed row-class order when W divides the row count. In prime radix q_{rad}, take a target $y \in [q_{\mathrm{rad}}^b]$ following a fixed number of $O(b)$-digit control fields. Across arbitrary spectators, it admits any $b^{O(1)}$-computable modular offset from those controls. It also admits multiplication modulo q_{rad}^b by a fixed rational whose numerator and denominator are coprime to the radix. Fixed-width signed addition, subtraction, sign changes, guarded binary shifts, bit reversal inside words, and reversal of record order cost $O(V)$.*

Long records absorb the per-record scalar work.

Lemma 3.4 (Long-record scans)**.** *For fixed $C \geq 0$, $O(q^C)$ scalar or header work per record, together with a scan, costs $O(Mq^C + V) = O(V)$ whenever $r \geq q^{C+1}$.*

Proof. Since $q, w \geq 1$, $q^C \leq r/q \leq rw$, so $Mq^C \leq Mrw \leq V$. The scan costs $O(V)$ by Fact 3.3. □

Fact 3.5 (Address-field interchange [16, Lemma 12])**.** *With $\tau = 1 - 2^{-50}$, exchanging two disjoint u-bit address fields across arbitrary prefix, gap, and suffix spectators costs $O(V(1 + u^\tau))$ on a fixed number of tapes.*

A *chunk* is a block of K consecutive address bits. A *slice* selects a common offset in D consecutive chunks. The next primitive acts on the whole slice, not a chosen subset.

Fact 3.6 (Parallel butterflies [16, Proposition 14])**.** *Fix $\beta = \frac{1}{2}$, $\tau = \sigma = 1 - 2^{-50}$, and $C_1 = 20$. Let positive rationals $c, \varepsilon, \lambda, \lambda'$ satisfy $\varepsilon C_1 < 1$ and*

$$\max\{\tau, \sigma, \tau(1 + c/\beta)\} < \lambda < 1, \tag{3}$$

$$\max\{\lambda, \sigma + \beta(1 - \sigma)\} < \lambda' < 1. \tag{4}$$

Here λ is an auxiliary recursion exponent. Fix positive range (band) constants $a_d \leq b_d$, $a_r \leq b_r$, C_M, C_{desc}, and $C_w > 1$. There are constants $q_{\min}, C_0$ and one fixed-tape procedure for all integer parameters satisfying

$$q \geq q_{\min}, \quad a_d q^\varepsilon \leq d \leq b_d q^\varepsilon, \quad 2^{a_r q/d} \leq r \leq 2^{b_r q/d}, \quad K = \lfloor d^c \rfloor. \tag{5}$$

Let $1 \leq D \leq d$, $A, S \geq 1$, $0 \leq \rho < K$, $M = AS2^{KD}$, $\log_2(2M) \leq C_M q$, and $q + 2 \leq w \leq C_w q$. The input is an array z of $\mathbb{D}_q^r$ records on $[A] \times [2^K]^D \times [S]$, of volume $V = 2Mrw$, with header

$$\Gamma(q)\Gamma(d)\Gamma(r)\Gamma(D)\Gamma(A)\Gamma(S)\Gamma(K)\Gamma(\rho + 1)\Gamma(w)$$

of length at most $C_{\mathrm{desc}} q$. Let H apply $H_0(u, v) = ((u + v)/2, (u - v)/2)$ to bit ρ, counted from the least significant end, in every chunk. The procedure evaluates Hz without internal rounding, using $\Delta = \lceil C_0 d^{C_1} \rceil = o(q)$ temporary guard bits: $q + \Delta$ fractional bits and $\Delta + 3$ integer/sign bits. It truncates once at the output boundary, returning $Q_q(Hz) \in \mathbb{D}_q^{Mr}$ at width w in $O(Vd^{\lambda'})$ steps, with $\|Q_q(Hz) - Hz\|_\infty < \sqrt{2}\, 2^{-q}$. The recursion stops at widths $e < d^\beta$, with $e = 1$ always a leaf. Precision q and prefix range A are denoted p and P in [16].

The formalization in Theorem 1.2 proves these primitives and both $h = 100$ networks from scratch. Using the exact edge-rank sums of [16, Propositions 7–8], it certifies $(\beta, C_1, 1 - \tau, 1 - \sigma) = (1/64, 14, 2^{-40}, 2^{-42})$.

4 The Fractional-Shift Bridge

Descending-axis spreading and ascending-axis cyclic solves turn one tensor DFT into a length-R DFT on the same grid. We prove the accuracy guarantee used in multiplication, Theorem 4.6.

4.1 Mixed-Radix Indices and the Half-Bin Kernel

Let $R = \prod_{a=1}^{d} t_a = 2^k$, where $d \geq 2$ and every $t_a \geq 2$ is a power of two. Put $A_a = \prod_{i<a} t_i$ for the input stride and define

$$j(\mathbf{j}) = \sum_{a=1}^{d} j_a A_a, \qquad f(\mathbf{u}) = \sum_{a=1}^{d} u_a \frac{R}{A_{a+1}}, \qquad \delta_a(\mathbf{j}) = \frac{\sum_{i<a} j_i A_i}{A_a} \in [0,1).$$

Here $j_a, u_a \in [t_a]$ index input and frequency axes; the fractional shift δ_a is used for $a \geq 2$. With $j = j(\mathbf{j})$, expand and drop integer terms from later input axes:

$$\frac{j f(\mathbf{u})}{R} \equiv \frac{j_1 u_1}{t_1} + \sum_{a=2}^{d} (j_a + \delta_a(\mathbf{j})) \frac{u_a}{t_a} \pmod 1.$$

Identify $x_{\mathbf{j}} = x_{j(\mathbf{j})}$. Increasing input position has field order $(j_d, \ldots, j_1)$; reversing these fields, but not their bits, gives tensor order. Natural output order $(u_1, \ldots, u_d)$ already agrees with increasing f.

Damping sinc localizes interpolation; the carrier phase avoids the midpoint (Nyquist) tie, as explained in Section 2.

Definition 4.1 (Negative half-bin carrier)**.** For a tail parameter $\omega \geq 8$, take $\alpha > 0$ with $\alpha^2 = 4\omega \ln 2/\pi$ and put $\alpha_\pi = \sqrt{\pi}\,\alpha$. Use $\operatorname{sinc}(s) = \sin(\pi s)/(\pi s)$, with $\operatorname{sinc}(0) = 1$, and $\widehat{\phi}(\xi) = \int_{\mathbb{R}} \phi(s) e^{-2\pi i \xi s}\, ds$. The error function is $\operatorname{erf}(z) = (2/\sqrt{\pi}) \int_0^z e^{-s^2}\, ds$. For $a \geq 2$, define

$$g_a(s) = e^{-\pi i s/t_a} \operatorname{sinc}(s) e^{-\pi s^2/\alpha^2}, \qquad \widehat{g}_a(\xi) = G\left(\xi + \frac{1}{2t_a}\right),$$

$$G(\xi) = \int_{\xi-1/2}^{\xi+1/2} \alpha e^{-\pi \alpha^2 s^2}\, ds = \frac{1}{2}\left[\operatorname{erf}\left(\alpha_\pi(\xi + \tfrac{1}{2})\right) - \operatorname{erf}\left(\alpha_\pi(\xi - \tfrac{1}{2})\right)\right],$$

$$H_\delta^{(a)}(e) = \sum_{s \in e + t_a\mathbb{Z}} g_a(s + \delta).$$

The box transform of sinc and the Gaussian transform give $\widehat{g}_a$. The folded kernel $H_\delta^{(a)}$ wraps the shifted kernel around a t_a-cycle. The carrier changes sign under a shift of t_a, so it is included before folding.

4.2 Exact Aliases and Conditioning

Define S_a to spread samples and $\mathcal{A}_a$ to mix shifted frequency copies, or *aliases*:

$$(S_a V)_{\ldots,k_a,\ldots} = \sum_{j_a < t_a} V_{\ldots,j_a,\ldots} H^{(a)}_{\delta_a(\mathbf{j})}(j_a - k_a),$$

$$(\mathcal{A}_a Z)_f = \sum_{\nu \in \mathbb{Z}} G\left(\frac{u_a(f) + 1/2}{t_a} - \nu\right) Z_{f - \nu R/A_a}.$$

Frequency subscripts are modulo R, and $u_a(f)$ is the ath digit of f. The shift δ_a is constant along a spread line. An alias shift changes only $u_1, \ldots, u_{a-1}$, giving the triangular factorization below.

Lemma 4.2 (Triangular alias identity)**.** *For $U^{\mathrm{ex}} = S_2 \cdots S_d x$, identifying $\mathbf{u}$ with $f(\mathbf{u})$ gives $\mathcal{F}_{\mathbf{t}} U^{\mathrm{ex}} = \mathcal{A}_2 \cdots \mathcal{A}_d (\mathcal{F}_R x)$. Operators compose from right to left.*

Proof. For a Schwartz function ϕ (smooth, with all derivatives rapidly decreasing), Poisson summation states

$$\sum_{e \in \mathbb{Z}} \phi(z-e) e^{2\pi i e \xi} = \sum_{\nu \in \mathbb{Z}} \widehat{\phi}(\xi - \nu) e^{2\pi i z (\xi - \nu)}. \tag{6}$$

Apply this with $\phi = g_a$, $z = j_a + \delta_a$, and $\xi = u_a/t_a$, unfolding $H_\delta^{(a)}$ on the left. Descending spreading leaves earlier digits fixed. Induction over $a = d, \ldots, 2$ gives the coefficient $\prod_{a=2}^{d} H_{\delta_a(\mathbf{j})}^{(a)}(j_a - k_a)$ of $x_{\mathbf{j}}$ in $U^{\mathrm{ex}}_{j_1,k_2,\ldots,k_d}$. After (6), its phase is $jf/R - \sum_{a=2}^{d} \nu_a j / A_a$ modulo one, since $\delta_a \equiv j/A_a \pmod 1$. Gaussian decay makes both sums absolutely convergent. Thus, with $Z = \mathcal{F}_R x$, interchanging sums gives

$$(\mathcal{F}_{\mathbf{t}} U^{\mathrm{ex}})_{\mathbf{u}} = \sum_{\boldsymbol{\nu} \in \mathbb{Z}^{d-1}} \left[\prod_{a=2}^{d} G\left(\frac{u_a + 1/2}{t_a} - \nu_a \right) \right] Z_{f(\mathbf{u}) - \sum_{a=2}^{d} \nu_a R/A_a}.$$

In $\mathcal{A}_2 \cdots \mathcal{A}_d$, a left factor $\mathcal{A}_b$ with $b < a$ shifts only earlier digits, leaving u_a fixed. Each coefficient is therefore evaluated at its original digit, giving this expansion. □

Keep only aliases $0, 1$: for $y = (u_a(f) + 1/2)/t_a$, put $c_0 = G(y)$, $c_1 = G(y-1)$, and $(B_a Z)_f = c_0 Z_f + c_1 Z_{f - R/A_a}$. Each cycle has even length $L = A_a$ and constant coefficients. Their gap, not L, controls the inverse.

Lemma 4.3 (Alias tails and conditioning)**.** *Let $\omega \geq 8$ and $t = t_a$. Then $\|\mathcal{A}_a\|_\infty, \|B_a\|_\infty \leq 1$, $c_{\max} := \max(c_0, c_1) \geq \frac{1}{2} - 0.060\, 2^{-4\omega} > 0.49999999$, and*

$$\sum_{\nu \notin \{0,1\}} G(y - \nu) < 0.120\, 2^{-\omega}, \qquad |c_0 - c_1| \geq \frac{7}{4t}.$$

On a cycle of any length $L \geq 1$, the inverse norm is at most $1/|c_0 - c_1| \leq 4t/7$, with equality in the first bound for even L. Thus $\|(B_2 \cdots B_d)^{-1}\|_\infty \leq (4/7)^{d-1}(R/t_1) \leq R/t_1$. For $Y = \mathcal{F}_{\mathbf{t}} U^{\mathrm{ex}}$,

$$\left\| (B_2 \cdots B_d)^{-1} Y - \mathcal{F}_R x \right\|_\infty \leq 0.120 (d-1) \frac{R}{t_1} 2^{-\omega} \|x\|_\infty . \tag{7}$$

Proof. The positive weights $G(y - \nu)$ sum to one because their intervals tile $\mathbb{R}$, so both operator norms are at most one. For $\operatorname{erfc}(s) = 1 - \operatorname{erf}(s)$, the omitted mass satisfies

$$\begin{aligned} \Psi(y) &= \tfrac{1}{2} \operatorname{erfc}(\alpha_\pi (y + \tfrac{1}{2})) + \tfrac{1}{2} \operatorname{erfc}(\alpha_\pi (\tfrac{3}{2} - y)) \leq \Psi(0), \\ \Psi(0) &\leq \frac{2^{-\omega}}{2\sqrt{\pi \omega \ln 2}} + \frac{2^{-9\omega}}{6\sqrt{\pi \omega \ln 2}} < 0.120\, 2^{-\omega}. \end{aligned}$$

The first inequality uses symmetry and convexity on $[0, 1]$. The second uses the integral bound $\operatorname{erfc}(s) \leq e^{-s^2}/(\sqrt{\pi} s)$ for $s > 0$. Evenness and differentiation show that G decreases on $[0, \infty)$, so $c_{\max} \geq G(1/2) = 1/2 - \operatorname{erfc}(\alpha_\pi)/2$. The same tail bound gives $\operatorname{erfc}(\alpha_\pi)/2 < 0.060\, 2^{-4\omega}$. The constant series in Section A certify $\pi > 3.14$ and $\ln 2 > 0.693$, which give the decimals.

For separation, set $\Delta_G(e) = G(1/2 - e) - G(1/2 + e)$ for $0 \leq e \leq 1/2$. Differentiation gives

$$\Delta_G'(e) = 2\alpha e^{-\alpha_\pi^2 e^2} \left(1 - e^{-\alpha_\pi^2} \cosh(2\alpha_\pi^2 e) \right).$$

Algorithm 2 Fixed-point BRIDGE(x)

Input: $x \in \widetilde{\mathbb{C}}^R_{p+2}$, $\|x\|_\infty \le 2^p$, with the parameters of Theorem 4.6. **Output:** $\widehat{\mathcal{F}}_R x \in \widetilde{\mathbb{C}}^R_{p+2}$.

1: Choose H from component magnitudes with $X \le 2^H < 4X$. Set $S = 2^{H+\lceil \lg(R/t_1)/2 \rceil + 4}$ and form $x' = x/S$ exactly on $\widetilde{\mathbb{C}}_q$.
2: Reverse the input axis fields. For $a = d, \dots, 2$, expose axis a as innermost, convolve its lines with $H^{(a)}_{\delta_a}(-e)$, and restore the order. Retain only unfolded indices $|s| \le P$ before folding. Use certified taps and signed Kronecker convolution, that is, integer products of packed coefficient blocks (Lemma 4.5). Apply $\mathcal{Q}_q$ once per completed entry. Call the result $\widehat{U}$.
3: Compute $\widehat{Y} = \mathcal{M}(\widehat{U})$.
4: For $a = 2, \dots, d$, apply the two-pass solver for B_a, using the stored coefficients $\hat{\mu}, \hat{\rho}, \hat{\gamma}$ and rounding $\mathcal{Q}_q$ specified in Lemma 4.5(iv).
5: Multiply by S exactly, apply $\mathcal{Q}_{p+2}$ once, and return.

The subtracted factor is at most $(1 + e^{-2\alpha_\pi^2})/2 < 1$; for $e \le 1/4$, it is at most $2^{-2\omega} \le 2^{-16}$. Thus Δ_G increases, and integration gives $\Delta_G(e) \ge (1 - 2^{-16}) \operatorname{erf}(\alpha_\pi e)$ for $e \le 1/4$. Every sampled bin has $|y - 1/2| \ge 1/(2t)$. Since $1/(2t) \le 1/4$, concavity gives $\operatorname{erf}(\alpha_\pi/(2t)) \ge (2/t) \operatorname{erf}(\alpha_\pi/4)$. Together with monotonicity, this yields

$$t\,|c_0 - c_1| \ge 2(1 - 2^{-16}) \operatorname{erf}(\alpha_\pi/4) > 2(1 - 2^{-16})\frac{22}{25} > \frac{7}{4}.$$

Indeed, $\alpha_\pi/4 \ge \sqrt{2 \ln 2}$, and the tail estimate gives $\operatorname{erf}(\alpha_\pi/4) > 1 - (4\sqrt{2\pi \ln 2})^{-1} > 22/25$.

Let $(TZ)_j = Z_{(j-1) \bmod L}$ and $\rho = \min(c_0, c_1)/c_{\max} < 1$. Dividing by the larger coefficient and applying T^{-1} if needed reduces the system to $I + \rho S$, with $S = T$ or T^{-1}. Since $S^L = I$, its inverse is $[1 - (-\rho)^L]^{-1} \sum_{j<L} (-\rho S)^j$. The powers occupy distinct columns in each row, giving inverse row norm

$$\frac{1 - \rho^L}{c_{\max}(1 - \rho)\,|1 - (-\rho)^L|} \le \frac{1}{|c_0 - c_1|},$$

with equality for even L. Multiply over axes for the product bound. Finally, telescoping gives

$$\mathcal{A}_2 \cdots \mathcal{A}_d - B_2 \cdots B_d = \sum_{a=2}^{d} B_2 \cdots B_{a-1}(\mathcal{A}_a - B_a)\mathcal{A}_{a+1} \cdots \mathcal{A}_d.$$

All other factors are contractions, so this difference has norm at most $0.120(d-1)2^{-\omega}$. Apply it to $\mathcal{F}_R x$ and then apply the inverse product; Lemma 4.2 gives (7). □

A positive carrier places $u_a = 0$ at $y = -1/(2t_a)$. For $t_a = 2$, the discarded $\nu = -1$ alias is $G(3/4)$. Integration over $[1/4, 3/8]$ gives $G(3/4) \ge (\alpha/8)2^{-9\omega/16}$, not $O(2^{-\omega})$.

For a real kernel, Hermitian symmetry forces $|\widehat{g}(1/2)| = |\widehat{g}(-1/2)|$, so a midpoint sample has no nonzero magnitude gap. Section 2 explains the resulting $\Theta(dk)$-bit stagewise loss for bounded weights.

4.3 Fixed-Point Implementation and Error

Henceforth $t_d = r = 2^\ell$, $\ell \ge 2$, and $t_i \in \{r/2, r\}$ for $i < d$. Put $q = P + 2$, $v = 2^{-q}$, and $X = \max(1, \|x\|_\infty)$. The approximate tensor transform $\mathcal{M} : \mathbb{D}^R_q \to \widetilde{\mathbb{C}}^R_q$ returns at width $w = q + 2$ with $\|\mathcal{M}(U) - \mathcal{F}_{\mathbf{t}} U\|_\infty \le E_{\mathrm{mid}} v$, where $0 \le E_{\mathrm{mid}} \le 16(k+1)$. Corollary 6.5 supplies $E_{\mathrm{mid}} = 12\ell$. In scan bounds, V uses the current working width.

We make each axis contiguous by swapping its field with the innermost one.

Lemma 4.4 (Axis exposures)**.** *Exposing an axis as innermost and restoring it costs $O(V(1+\ell^\tau))$. All spread exposures and the initial field reversal cost $O(Vd(1+\ell^\tau))$.*

Proof. Unchanged axes form prefix, gap, and suffix spectators.

- Equal-width fields use Fact 3.5.
- For widths differing by one bit, write the fields as (x_0, X) and Y, with gap G and equal-width X, Y. Use
$$(x_0, X, G, Y) \longmapsto (x_0, Y, G, X) \longmapsto (Y, G, x_0, X),$$
or its inverse.

The swap has width $\ell - 1$ or ℓ; moving x_0 costs $O(V)$ by Fact 3.3. Exposure and restoration use at most $2(d-1)$ exchanges, and initial reversal uses at most $\lfloor d/2 \rfloor$. We retain original digit labels when computing δ_a. □

The next bounds account for spreading, packed convolution, and cyclic solves.

Lemma 4.5 (Local fixed-point bounds)**.** *Let $P \geq 20$, $q = P + 2$, $v = 2^{-q}$, $8 \leq \omega \leq P$, and $t = t_a$.*

(i) *For every $\delta \in [0,1)$, $\left\|H_\delta^{(a)}\right\|_2 \leq 1$, $\left\|H_\delta^{(a)}\right\|_1 \leq \sqrt{t}$, and $\sum_{|e|>P} |g_a(e+\delta)| < 0.40v$.*

(ii) *On $\widetilde{\mathbb{C}}_q^t$ inputs of norm at most one, truncated folded taps certified within $2v$ give spreading error $< 4(P+1)v$. Put $W_t = \min(2P+1, t)$ and $B_0 = 2^{4P+16}$. A line uses at most $2t/W_t$ blocks, each requiring four integer products with operands of at most $W_t(4P+16)+1$ bits, plus $O(tP)$ total bit work.*

(iii) *For $\delta \in A_a^{-1}\mathbb{Z} \cap [0,1)$, retained kernel samples, alias weights, $\mu = c_{\max}^{-1}$, and $\rho = \min(c_0, c_1)/c_{\max}$ can be certified within 2^{-b} in $O((b+k)^3)$ time for $b \geq P$. The same holds for phases with $O(k + \lg P)$-bit rational angles. Folded q-grid taps within v cost $O(P(P+k)^3)$ each. Real parameters can be stored within v, with $\hat{\rho} \geq 0$; unit phases can be stored in $\mathbb{D}_q$ within $2v$.*

(iv) *Let $L \geq 2$ be even and $v \leq 1/(64t)$. Suppose $\hat{\mu}, \hat{\rho}, \hat{\gamma} \in 2^{-q}\mathbb{Z}$ are within v of $\mu, \rho, (1-\hat{\rho}^L)^{-1}$, respectively, with $\hat{\rho} \geq 0$. For $Y \in \widetilde{\mathbb{C}}_q^L$ and $Y_* = \max(1, \|Y\|_\infty)$, the two-pass solver for $B = c_0 I + c_1 T$ has error at most $32t^2 v Y_*$ and states of modulus $< 7tY_*$. All cycles have $O(V)$ scan overhead per axis, without transposition.*

(v) *Binary powering truncated at $Q = q + \lg L + 2\lg(4t) + 10$, followed by guarded division and rounding, produces such a $\hat{\gamma}$. For $L \leq R$, its scalar arithmetic uses $O(P+k)$ bits.*

Proof. For (i), (6) expresses the unnormalized Fourier coefficients of $H_\delta^{(a)}(-\cdot)$ as $\sum_\nu G((u+1/2)/t - \nu) e^{2\pi i \delta(u/t-\nu)}$. The weights sum to one, so each coefficient has modulus at most one. Parseval gives $\left\|H_\delta^{(a)}\right\|_2^2 \leq t^{-1}\sum_{u<t} 1 = 1$; Cauchy–Schwarz gives the ℓ_1 bound. Also $|g_a(s)| \leq e^{-\pi s^2/\alpha^2}/(\pi\,|s|)$. Each omitted tail has unit spacing beyond P, and $\pi P^2/\alpha^2 \geq P \ln 2$ since $\omega \leq P$. Bounding each tail by its first term plus an integral gives

$$\sum_{|e|>P} |g_a(e+\delta)| \leq \left(\frac{2}{\pi P} + \frac{1}{\pi P \ln 2}\right) 2^{-P} < 0.06\, 2^{-P} = 0.24v < 0.40v.$$

For the integral, use $z = \pi s^2/\alpha^2$ and $\int_b^\infty e^{-z} z^{-1}\, dz \leq e^{-b}/b$.

For (ii), at most $2P+1$ retained coefficients contribute $(4P+2.4)v$ error, including the tail; final rounding adds less than v. After cyclic reindexing, the retained kernel occupies W_t consecutive slots. Split the data into blocks of at most W_t entries.

By (i), data and stored-tap numerators have modulus $< 2 \cdot 2^q$. Thus packing $\sum_{j<W_t} u_j B_0^j$ uses at most $W_t(4P+16)+1$ signed bits. Four real products give each complex block convolution. Every product coefficient, signed combination, and overlap sum has modulus at most $16(2P+1)2^{2q} < B_0/2$. Hence extracting residues in $[-B_0/2, B_0/2)$ recovers the signed coefficients exactly. Adjacent blocks overlap, and the final linear convolution wraps at most once modulo t. Packing, extraction, overlap-add, reindexing, and rounding cost $O(tP)$. There are $\lceil t/W_t \rceil \le 2t/W_t$ blocks.

For (iii), apply Lemma A.1. Evaluate the at most $2P+1$ samples of a folded tap at $b = q + \lg(2P+2) + 12$ and sum at a common dyadic scale. Evaluation, nearest rounding, and the tail from (i) contribute less than $(2^{-12} + 1/\sqrt{2} + 0.24)v < v$. Rational enclosures also give the stored reals and inward-scaled disk phases. No transcendental rounding oracle is used.

For (iv), orient the recurrence according to the larger coefficient:

- If $c_0 > c_1$, use $(TZ)_j = Z_{(j-1) \bmod L}$.
- If $c_1 > c_0$, set $z_i = Z_{(-i) \bmod L}$ and $y'_i = Y_{(1-i) \bmod L}$, and use y' as the right-hand side.

The test $u_a < t_a/2$ decides the orientation exactly. Write Y for the oriented right-hand side. A pass maintains z and computes, in order,

$$b_j = \mathcal{Q}_q(\hat{\mu} Y_j), \qquad z \leftarrow b_j + \mathcal{Q}_q(-\hat{\rho} z).$$

Pass 1 starts at zero. Close its final state by $\widehat{Z}_{L-1} = \mathcal{Q}_q(\hat{\gamma} z)$, then start pass 2 from this seed and compute indices $0, \dots, L-2$.

The hypotheses and Lemma 4.3 give $\rho \le 1 - 7/(4t)$, $0 \le \hat{\rho} \le 1 - 1/(2t)$, and $|\hat{\mu}| < 2.01$. Set $J_s = (I + sT)^{-1}$ and let $\widetilde{Z} = \hat{\mu} J_{\hat{\rho}} Y$ be the exact solution with stored coefficients. Then $\|J_\rho\|_\infty \le t$, $\|J_{\hat{\rho}}\|_\infty \le 2t$, and $J_{\hat{\rho}} - J_\rho = (\rho - \hat{\rho}) J_{\hat{\rho}} T J_\rho$. Using $\|B^{-1}\|_\infty \le 4t/7$ gives

$$\left\| \widetilde{Z} - B^{-1} Y \right\|_\infty \le (2t + 8t^2/7) v \, \|Y\|_\infty \le 6t^2 v \, \|Y\|_\infty .$$

Each step adds rounding error $< 2v$. Put $S_L = \sum_{j<L} \hat{\rho}^j$. Unrolling the first pass gives unrounded endpoint $(1 - \hat{\rho}^L)\widetilde{Z}_{L-1}$, since L is even. Its rounding error is at most $2vS_L$, so exact closure contributes $2vS_L/(1-\hat{\rho}^L) = 2v/(1-\hat{\rho}) \le 4tv$, independently of L. Prefix states have modulus at most $(2.01Y_* + 2v)/(1-\hat{\rho}) < 4.1tY_*$. Closing with $\hat{\gamma}$ and rounding therefore give $\left| \widehat{Z}_{L-1} - \widetilde{Z}_{L-1} \right| \le 12tvY_*$.

Pass 2 contracts the seed error and adds at most $4tv$ in new rounding error, yielding total error $(6t^2 + 16t)vY_* \le 32t^2 vY_*$. Finally, $\left\| \widetilde{Z} \right\|_\infty < 4.02tY_*$ and $v \le 1/(64t)$ bound all states by $7tY_*$.

For (v), truncated binary powering underestimates $\hat{\rho}^L$. Induction on the exponent, using $|a^2 - b^2| \le 2|a-b|$ on $[0,1]$, gives error at most $2L2^{-Q}$. Since $1 - \hat{\rho}^L \ge 1/(2t)$, the reciprocal changes by at most $8t^2 L 2^{-Q} \le v/2048$. Division within $v/4$, followed by nearest rounding to $2^{-q}\mathbb{Z}$, gives error below v. Since $\lg L, \lg t \le k$, all scalar widths are $O(P+k)$.

For streaming, write $f = j(R/A_a) + s$ and store one $(z, \hat{\mu}, \hat{\rho}, \hat{\gamma})$ record per column s on a side tape. Each row scan advances every cycle once. Its R/A_a states are rewound after each of A_a rows, costing $O(V)$ per pass; the main-tape rewind between passes costs the same. Save closing values before pass 2 overwrites states. Split on the leading bit of u_a to separate orientations. Reverse cycles use row orders $j \mapsto -j$ and $j \mapsto 1-j$, implemented by row-block reversals and cyclic shifts in $O(V)$ time by Fact 3.3; merge afterward. Scalar arithmetic and table construction are charged separately. □

The inverse-norm product R/t_1 makes the required guard precision depend on k, not dk.

Theorem 4.6 (Finite-precision bridge accuracy)**.** *For the balanced shape above, let $p \ge \max(8,k)$ and $x \in \widetilde{\mathbb{C}}^R_{p+2}$ satisfy $\|x\|_\infty \le 2^p$. Set $X = \max(1, \|x\|_\infty)$ and*

$$\begin{aligned} P &= 2p + 12k + 32\lg(p+k+2) + 256, \\ \omega &= p + k + \lg d + 20, \qquad q = P + 2, \qquad w = q + 2. \end{aligned} \tag{8}$$

Use $\alpha^2 = 4\omega \ln 2/\pi$, real table errors at most $v = 2^{-q}$, complex tap and disk-phase errors at most $2v$, and closure precision Q from Lemma 4.5. For any middle procedure specified above, Algorithm 2 returns

$$\begin{aligned} \left\| \widehat{\mathcal{F}}_R x - \mathcal{F}_R x \right\|_\infty &< \left(\frac{\sqrt{2}}{8} + 2^{-12} + 2^{-23} \right) 2^{-p} X \\ &< 0.177021\, 2^{-p} X < \frac{1}{4}\, 2^{-p} X. \end{aligned} \tag{9}$$

Before final rounding, arithmetic error is at most $2^{-p-12}X$ and omitted-alias error is at most $2^{-p-23}X$. Outside the middle procedure, scalar arithmetic uses $O(P)$ bits and packed Kronecker operands use $O(P^2)$ bits, with $P = O(p+k)$.

Proof. With $z = p + k + 2 \ge 12$ and $b_0 = \lg z$, the schedule gives $P + 1 \le 44z + 233 \le 64z$ and $P > p + 4k + b_0 + 30$. Since $d \le k \le 2^k$, we have the common precision margin

$$2^{14} d(P+1) R^3 v \le 2^{18+4k+b_0-P} \le 2^{-p-12}.$$

The schedule also gives $16d(P+1)\sqrt{R/t_1}v < 1/16$, $E_{\mathrm{mid}} v < 1/100$, $v \le (64 \max_a t_a)^{-1}$, $64v\sum_{a=2}^d t_a < 1$, and $8 \le \omega \le P$.

Compute $C = \max(1, \max_j |\mathrm{Re}\, x_j|, \max_j |\mathrm{Im}\, x_j|)$ and take $H = 1 + \lceil \log_2 C \rceil$. Then $H \le p+1$, $X \le 2^H < 4X$, and $16\sqrt{R/t_1}X \le S < 128\sqrt{R/t_1}X$. The quotient x/S needs at most $2p + \lceil k/2 \rceil + 7 \le q$ fractional bits, so $x' = x/S$ is exact on $\widetilde{\mathbb{C}}_q$. Since $\|S_a\|_\infty \le \sqrt{t_a}$, every exact partial spread has norm at most $1/16$.

Let $U = S_2 \cdots S_d x'$. Errors from axis a pass through the remaining smaller-axis spreads, so

$$E_{\mathrm{spread}} := \left\| \widehat{U} - U \right\|_\infty \le 4(P+1)v \sum_{a=2}^{d} \prod_{j=2}^{a-1} \sqrt{t_j} \le 16d(P+1)\sqrt{R/t_1}v < \frac{1}{16}.$$

The same estimate inductively bounds every computed partial spread by $1/8$, validating the local bounds and middle call. Contraction of $\mathcal{F}_{\mathbf{t}}$ gives $E_Y := \left\| \widehat{Y} - \mathcal{F}_{\mathbf{t}} U \right\|_\infty \le E_{\mathrm{spread}} + E_{\mathrm{mid}} v \le 32d(P+1)Rv$. Also $\left\| \widehat{Y} \right\|_\infty < 1/8 + 1/100 < 1$.

For the solves, put $R_1 = 1$ and $R_a = \prod_{j=2}^a t_j$. A computed solve multiplies $\max(1, \|\cdot\|_\infty)$ by at most $t_a(1 + 32 t_a v)$. Since $1 + s \le e^s$ and $32v\sum_{a=2}^d t_a < 1/2 < \ln 2$, the product through axis a is below $2R_a$. Axis a therefore has input bound $2R_{a-1}$ and local error at most $64 t_a^2 R_{a-1} v$. Propagating these errors through the remaining exact inverses gives

$$E_{\mathrm{solver}} \le 64v \sum_{a=2}^{d} t_a^2 R_{a-1} \frac{R_d}{R_a} = 64 \frac{R}{t_1} v \sum_{a=2}^{d} t_a \le 64 d R^2 v.$$

Before rescaling, arithmetic error is at most $(R/t_1)E_Y + E_{\mathrm{solver}} \le 96d(P+1)R^2 v$. Since $S < 128RX$, rescaling gives error $< 2^{14} d(P+1) R^3 v X \le 2^{-p-12} X$.

Apply (7) to x' and multiply by S. Since $S\|x'\|_\infty = \|x\|_\infty \le X$, the omitted-alias error is at most $0.120(d-1)(R/t_1)2^{-\omega}X < 0.120\, 2^{-p-20} X < 2^{-p-23}X$. Final nearest rounding adds at most $(\sqrt{2}/8)2^{-p}$; summing and using $X \ge 1$ proves (9).

Solver states on axis a have modulus below $14R_a \le 14R$, so $q+k+6$ signed bits suffice. Products with $2q$ fractional bits, closure arithmetic, table evaluation, and rescaling use $O(P)$-bit scalars. Kronecker operands have $O(P)$ slots of $4P+16$ bits each. □

Each spread axis has at most $2R/r$ lines, so regenerating all taps costs $O(dR(P+k)^{O(1)}/r)$. Solver tables have at most r parameter choices per axis; $R \ge t_1 r \ge r^2/2$ puts their construction under the same bound. For the family in Lemma 7.1, r dominates every fixed polynomial in P, so Lemma 3.4 absorbs this setup. Section 7.1 combines the remaining arithmetic and layout costs.

5 Exact Address-Bit Permutations from Butterfly Layers

We turn butterfly layers into exact record permutations for the tensor FFT, using the cancellations in Figure 3. Assume the admissible family of Fact 3.6, including (5). Let $M = 2^{m_0} = R/r$ length-r records have component width w and volume $V = 2Mrw$.

Number address positions from the most significant end. A chunk contains K consecutive positions, and a slice selects one common least-significant-bit offset in $D \le d$ consecutive chunks. Its layer applies $H_0(u,v) = ((u+v)/2, (u-v)/2)$ on every slice bit, then truncates once by Q_q. The following identity cancels the action on unselected bits.

Lemma 5.1 (Three-layer selection)**.** *Partition a D-bit slice into T, U, with $s = |T|$. Let H_{all} and H_T apply H_0 on the indicated bits and the identity elsewhere. Put $S = S_{\mathrm{ph}} = \mathrm{diag}(1, i)$ and $(S_U z)(x) = i^{\sum_{j\in U} x_j} z(x)$ for binary addresses x. Then*

$$H_0^3 = \tfrac{1}{2} H_0, \qquad (H_0 S)^3 = \tfrac{1+i}{4} I, \qquad H_T = 2^s [2(1-i)]^{D-s} (H_{\mathrm{all}} S_U)^3.$$

Proof. Matrix multiplication gives $H_0^2 = I/2$ and $(2H_0 S)^3 = 2(1+i)I$. Operators on distinct bits commute, so tensoring gives $(H_{\mathrm{all}} S_U)^3 = 2^{-s}((1+i)/4)^{D-s} H_T$. Since $(1-i)^2 = -2i$, writing $D - s = 2f + e$ with $e \in \{0,1\}$ gives the correction $2^{s+3f+e}(-i)^f (1-i)^e$. It uses a phase ± 1 or $\pm i$, at most one addition/subtraction pair $(a,b) \mapsto (a+b, b-a)$, and a binary shift. □

Target-bit butterflies turn a controlled sign into a controlled bit flip. For disjoint control–target pairs (c_j, t_j), put $T = \{t_1, \dots, t_s\}$ and $(\mathrm{CZ} z)(x) = (-1)^{\sum_j x_{c_j} x_{t_j}} z(x)$.

Lemma 5.2 (Controlled bit flips)**.** *The record permutation replacing each x_{t_j} by $x_{t_j} \oplus x_{c_j}$ is $\mathrm{CNOT}^{\otimes s} = 2^s H_T \mathrm{CZ} H_T$.*

Proof. We check the two values of the control bit.

- For control 0, the target map is $2H_0^2 = I$.
- For control 1, it is $2H_0 \,\mathrm{diag}(1,-1) H_0 = \left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$.

Tensoring over disjoint pairs proves the identity. Since the address map is its own inverse, indexing by old or new addresses gives the same operator. □

Appending zero fractional bits makes every layer boundary exact without leaving the primitive's size bands.

Lemma 5.3 (Exact boundaries and precision lifting)**.** *After enlarging the fixed bands, descriptor bound, and cutoff, a disk array on $\widetilde{\mathbb{C}}_q$ can be lifted in $O(V)$ time to $q_* = q + 8d$, $w_* = w + 8d$. Both three-layer selections in $H_T \mathrm{CZ} H_T$ are exact: their six boundary truncations discard only zeros. Their corrected outputs have entries on $\widetilde{\mathbb{C}}_{q+s}$ and $\widetilde{\mathbb{C}}_{q+2s}$. Every layer call receives and returns a disk array; lifted volume is $O(V)$ and guarded width is $O(q)$.*

Algorithm 3 Exact positional permutation PositionalPerm(π, z)

Input: A disk array of 2^{m_0} records and a permutation π of its address positions.

1: Factor $\pi = I_1 \circ I_0$ by cycle reflections and expand into six controlled-flip stages.
2: In each stage, group non-leftover targets by slice and apply Proposition 5.4. Handle each leftover target by moving control and target bits to the front, splitting into four streams, merging in XOR order, and undoing the moves (Fact 3.3).

Proof. A full D-bit layer adds at most D fractional bits, while fourth-root phases add none. Thus the first three boundaries need at most $q + jD$ bits, $j = 1, 2, 3$. The Gaussian-integer correction gives $H_T z$, needing at most $q + s$ bits. After CZ, the next three bounds are $q + s + jD \le q + 4d < q_*$. Since Fact 3.6 has no internal rounding, it truncates only zeros. The second correction needs at most $q + 2s$ bits; even the coarse bound $q + 6D$ fits the lift.

Normalized layers and fourth-root phases are contractions. Apply each correction in the order phase, multiplication by $1 - i$ if needed, then left shift. Moduli increase only toward the corrected output of a contraction, so all boundaries stay in the disk.

Since $d = O(q^\varepsilon)$ and $\varepsilon C_1 < 1$, eventually $q_* \le 2q$. Replacing a_d, a_r by $a_d/2, a_r/2$ covers the lower bands, since $q_*^\varepsilon \le 2q^\varepsilon$. The upper bands and address bound remain valid because $q_* \ge q$. Also $q_* + 2 \le w_* \le C_w q_*$ and $w_* = O(w)$. Only two header fields, precision and width, change, adding at most four bits. Appending zeros is a stream operation, and the guarded width $q_* + 2\lceil C_0 d^{C_1} \rceil + 3$ is $O(q)$. □

The sign pass reads restored bits, so controls need not be moved.

Proposition 5.4 (Slice-batched controlled flips)**.** *Suppose the targets of disjoint controlled flips lie in one slice. Controls may lie anywhere else, including unselected slice positions. Six layer calls at $q_* = q + 8d$ and $O(1)$ scans execute the batch exactly in $O(Vd^{\lambda'})$ time, returning to precision q and width w.*

Proof. Execute $2^s H_T \mathrm{CZ} H_T$ at lifted precision using Lemmas 5.1 and 5.3. Unselected bits are restored before CZ reads them, so they may be controls. The final 2^s scaling stays in the disk because its result is the input permuted. That result lies on $\widetilde{\mathbb{C}}_q$, so lowering is exact. The address counter and pair list give phases and parities in $q^{O(1)}$ time per record. The bands give $r/q^C \to \infty$ for every fixed C, so Lemma 3.4 absorbs this work into the scans. □

A cycle is a product of two reflections, each consisting of disjoint swaps.

Lemma 5.5 (Two-involution factorization)**.** *Every permutation π of address-bit positions factors as $I_1 \circ I_0$, where each I_a consists of disjoint swaps and fixed positions. Each involution uses three stages of disjoint controlled flips.*

Proof. On a cycle $i_j \mapsto i_{j+1}$, with indices modulo its length, set $I_0(i_j) = i_{-j}$ and $I_1(i_j) = i_{1-j}$. Both are involutions, with $I_1 I_0(i_j) = i_{j+1}$. A swap uses $(a, b) \mapsto (a \oplus b, b) \mapsto (a \oplus b, a) \mapsto (b, a)$. Disjoint swaps give disjoint updates in each stage, hence six stages overall. □

Put $J = \lfloor m_0/K \rfloor$. Group the first JK positions into chunks, then into consecutive groups of at most d chunks. A group starting at position j_0 has shape $[2^{j_0}] \times [2^K]^D \times [2^{m_0 - j_0 - DK}]$ and K slices. Hence $N_{\text{slice}} = K\lceil J/d \rceil \le m_0/d + K$; fewer than K positions remain.

Theorem 5.6 (Bit-exact positional permutations)**.** *Under the lifted admissible family, place a record in $\mathbb{D}_q^r$ at width w at every m_0-bit address, with $m_0 = O(q)$. With fixed interface bounds covering*

the slice headers, Algorithm 3 returns the induced record permutation at the original precision and width, with zero error, in

$$O\left(V\left[\left(\frac{m_0}{d}+K\right)d^{\lambda'}+K\right]\right)$$

steps. If $dK = o(q)$, this is $O(Vq\,d^{\lambda'-1})$.

Proof. Disjoint flips commute, so grouping a stage by target slice preserves it. The six stages use at most $6N_{\text{slice}}$ batches, hence $36N_{\text{slice}}$ full-layer calls, and fewer than $6K$ leftover flips of cost $O(V)$ each. Computing cycles, masks, and headers costs $q^{O(1)}$ setup time, absorbed by Lemma 3.4. Every operation is exact. The final bound uses $m_0 = O(q)$ and $K = o(q/d)$. □

These are permutations of address-bit positions, not arbitrary records. They include power-of-two matrix transposes, which exchange address fields. The method uses long arithmetic records and cancellation of linear combinations, so it gives no scalar-record bound in the transposition comparison of [9].

6 Direct Complex Tensor FFT

We compute the bridge's normalized tensor DFT using exact positional permutations, butterfly layers, and one combined phase per outer round. Use $t_d = r = 2^\ell$, $t_i \in \{r/2, r\}$ for $i < d$, and $R = \prod_i t_i$. Axis d is contiguous; write $\mathbf{t}_{<d} = (t_1, \dots, t_{d-1})$ for the outer axes, whose addresses have $m_0 = \log_2(R/r)$ bits.

Inward phase approximation keeps subsequent butterflies inside the unit disk.

Lemma 6.1 (Disk-valued phases)**.** *For $q \geq 8$ and $1 \leq \ell \leq q$, the phases $\theta_j = e^{\pi ij/r}$, $0 \leq j < 2r$, have deterministic approximations $\tilde{\theta}_j \in \mathbb{D}_q$ with $|\tilde{\theta}_j - \theta_j| < 2 \cdot 2^{-q}$. Fourth roots are exact, and each entry costs $O(q^3)$ bit operations.*

Proof. We treat fourth roots and all other phases separately.

- Return fourth roots exactly.
- Otherwise, Lemma A.1 at precision $q+5$ gives e_0 with $|e_0 - \theta_j| \leq 2^{-q-5}$ in $O(q^3)$ time. For $u_0 = 2^{-q}$, set $\tilde{\theta}_j = Q_q((1-u_0/8)e_0)$. Before truncation, the modulus is at most $(1-u_0/8)(1+u_0/32) < 1$ and the error is less than $u_0/4$. Truncation preserves the disk and adds less than $\sqrt{2}u_0$; $(\sqrt{2}+1/4)u_0 < 2u_0$ proves the claim.

□

Truncation preserves the disk of the exact product.

Lemma 6.2 (Disk-preserving complex products)**.** *For $a, b \in \mathbb{D}_q$, exact multiplication on $\widetilde{\mathbb{C}}_{2q}$ followed by Q_q returns a member of $\mathbb{D}_q$ with error less than $\sqrt{2}\,2^{-q}$. It uses four exact $(q+2)$-bit integer products and $O(q)$ additional work.*

Proof. For $a = a_0 + ia_1$, $b = b_0 + ib_1$, form both coordinates $(a_0b_0 - a_1b_1) + i(a_0b_1 + a_1b_0)$ at denominator 2^{2q} before truncating. Since $|ab| \leq 1$, (1) gives disk preservation and the error bound. □

Splitting the input halves gives the decimation-in-frequency (DIF) identity recalled in Section 2: for even u, $v \in [u/2]$, and $b \in \{0, 1\}$,

$$(\mathcal{F}_u z)_{2v+b} = \frac{1}{u/2}\sum_{j<u/2} e^{2\pi ivj/(u/2)} e^{2\pi ibj/u}\frac{z_j + (-1)^b z_{j+u/2}}{2}. \tag{10}$$

Algorithm 4 Direct complex tensor transform TensorFFT(z)

Input/output: Disk arrays on $\widetilde{\mathbb{C}}_q$, in natural tensor order, at width w.

1: Build the phase table. Run ℓ DIF levels within each record on axis d, using Q_q at butterfly boundaries and Lemma 6.2 for phases. Apply exact intra-record bit reversal.
2: Apply PositionalPerm($\mathcal{L}, z$).
3: For $v = \ell - 1, \dots, 0$, skip empty levels and apply the active outer butterflies. Use one parallel layer per bulk level. On the at most K boundary levels, use sequential exact butterflies at precision $q + d$, then truncate once to q. Compute J per record and multiply by $\tilde{\theta}_{2J}$ using Lemma 6.2.
4: Apply PositionalPerm($\mathcal{U}, z$).

Thus a round is a butterfly followed by an odd-branch phase. Processing input bits high to low reverses the frequency-bit order.

Write $a_i = \log_2 t_i$ for $i < d$, and name their bits $(i, a_i - 1), \dots, (i, 0)$, from most to least significant. The natural list $\mathcal{O}$ orders axes increasingly and bits decreasingly. Set $b_0 = (\ell - 1) \bmod K$ and $J_{\mathrm{blk}} = (\ell - 1 - b_0)/K$. The boundary list $\mathcal{P}$ contains the extra top bits $(i, \ell - 1)$ with $a_i = \ell$, then levels $b_0 - 1, \dots, 0$, each in increasing axis order. It has at most $(d-1)K$ bits. The remaining bits form the following list, where $\|$ denotes concatenation:

$$\mathcal{C} = \bigparallel_{j=0}^{J_{\mathrm{blk}}-1} \bigparallel_{i=1}^{d-1} ((i, \ell - 2 - jK), \dots, (i, \ell - 1 - (j+1)K)).$$

Let $\mathcal{L}$ move the named bits from $\mathcal{O}$ to $\mathcal{P} \,\|\, \mathcal{C}$, and let $B(i, v) = (i, a_i - 1 - v)$ reverse each original axis field. Each bulk level now selects one aligned bit per outer axis.

Lemma 6.3 (Chunked DIF layout)**.** *At each bulk level $b_0 \le v \le \ell - 2$, active bits occupy offset $\rho = (v - b_0) \bmod K$ in $d-1$ consecutive K-bit chunks. Exact outer DIF on $\mathcal{L}z$ returns $\mathcal{L}B\mathcal{F}_{\mathbf{t}_{<d}}z$. The single positional map $\mathcal{U} = (\mathcal{L}B)^{-1} = B^{-1}\mathcal{L}^{-1}$ restores natural order. Both maps have polynomial-time descriptions and are implemented by Theorem 5.6.*

Proof. Level v uses block $j = \lfloor (\ell - 2 - v)/K \rfloor$. Its lowest bit is $b_0 + (J_{\mathrm{blk}} - j - 1)K$, giving the stated offset. These chunks and their prefix and suffix spectators have shape $[A] \times [2^K]^{d-1} \times [S]$. In natural coordinates, (10) gives $B\mathcal{F}_{\mathbf{t}_{<d}}$. Conjugating each round by $\mathcal{L}$ gives $\mathcal{L}B\mathcal{F}_{\mathbf{t}_{<d}}\mathcal{L}^{-1}$; applying it to $\mathcal{L}z$ proves the claim. The lists and inverses use $m_0 = O(q)$ positions and take $q^{O(1)}$ work. □

We invert $\mathcal{L}B$, not $\mathcal{B}L$: B reverses original fields, whereas $\mathcal{L}$ moves bits out of them.

At level v, active axes have $a_i > v$ and subproblem length 2^{v+1}, which divides r. From their named slots read $b_i = x_{(i,v)}$ and $j_{i,<v} = \sum_{s<v} 2^s x_{(i,s)}$; inactive axes contribute zero. Axis i needs the phase $e^{2\pi i b_i j_{i,<v}/2^{v+1}}$, so all phases have common denominator r. Each phase depends only on its own axis bits and therefore commutes with other-axis butterflies. The round is $D_\zeta H$, where D_ζ multiplies record x by

$$J(x) = \sum_{i<d} b_i j_{i,<v} 2^{\ell-1-v} \pmod r, \qquad \zeta(x) = e^{2\pi i J(x)/r} = \theta_{2J(x)}.$$

Compute J exactly, then scan and rewind the $2r$-entry table once per record to retrieve $\tilde{\theta}_{2J}$. This costs $O(rq)$ per record, and the retrieved scalar multiplies all its entries. Combining exponents before approximation avoids a factor d in the round error.

Theorem 6.4 (Direct complex tensor FFT)**.** *Assume the admissible family of Fact 3.6, with the lifting convention of Lemma 5.3, $q \ge 8$, $d \ge 2$, $2 \le \ell < q$, $d\ell \le C_\ell q$, and $1 \le K \le \ell - 1$. Take*

$C_M \geq C_\ell + 1$, $C_{\mathrm{desc}} \geq 32(C_\ell + C_w + 1)$, *and* $q + 2 \leq w \leq C_w q$. *On every* $U \in \mathbb{D}_q^R$, *the algorithm returns a disk array in natural frequency order at unchanged width, with*

$$\|\textsc{TensorFFT}(U) - \mathcal{F}_{\mathbf{t}} U\|_\infty < 12\ell\, 2^{-q}, \qquad T = O\Big(V[dK + \ell d^{\lambda'} + \ell\, \mathsf{m}(q+2)]\Big),$$

where $V = 2Rw = \Theta(Rq)$.

Proof. The exact axis-d transform commutes with outer maps, which act identically on each position within a record. Thus (10) and Lemma 6.3 give $\mathcal{F}_{\mathbf{t}}$ in natural order.

Butterflies are contractions, stored phases have modulus at most one, and truncation preserves the disk. For exact and stored phase diagonals $D, \widetilde{D}$, butterfly truncation, phase approximation, and product truncation give

$$\Big\|Q_q(\widetilde{D} Q_q(Hz)) - DHz\Big\|_\infty < (2\sqrt{2} + 2)2^{-q} < 6 \cdot 2^{-q}.$$

Boundary and intra-record levels obey the same bound, since their butterflies are exact before one truncation. For an exact contractive round A, $\big\|\widehat{A}(z) - Az_0\big\|_\infty \leq \big\|\widehat{A}(z) - Az\big\|_\infty + \|z - z_0\|_\infty$. Summing over ℓ inner levels and at most ℓ outer rounds proves the bound, since permutations are exact. This uses no continuity of Q_q.

The phase table costs $O(rq^3) = o(V)$, since $M \geq r/2$ and the bands make r superpolynomial in q. At inner subproblem length u, a master-table scan extracts $(\tilde{\theta}_{(2r/u)j})_{j<u/2}$; replaying it over all blocks costs $O(Rq)$, including rewinds. Butterflies and products cost $O(Rq + R\mathsf{M}(q+2))$ per level. Elementary two-bit stream passes perform intra-record bit reversal in $O(V\ell)$.

The outer levels have two costs:

- Bulk rounds cost $O(V\ell d^{\lambda'})$.
- Each of the at most $b_0 + 1 \leq K$ boundary levels uses at most $d - 1$ ordinary passes. Exact values need at most $q + d - 1$ fractional bits, so addition guards still use $O(q)$-bit temporaries. These levels cost $O(VdK)$ in total.

Outer phase lookups cost $O(rq)$ per record, and Lemma 3.4 absorbs the polynomial computation of J. Their products total $O(R\ell\mathsf{M}(q+2))$. The two layouts cost $O(V\ell d^{\lambda'})$, since $m_0/d \leq \ell$ and $K < \ell$. Slice headers have length $2m_0 + 2\ell + O(\lg q)$, within the stated bounds. Summing proves the cost. Removing temporary lifts leaves every completed stage in $\mathbb{D}_q^R$ at width w. □

Thus the bridge needs no extra output precision.

Corollary 6.5 (Walsh middle transform)**.** *At bridge precision* $q = P + 2$ *and width* $w = q + 2$, *the direct transform meets Theorem 4.6 with* $E_{\mathrm{mid}} = 12\ell \leq 16(k+1)$ *and no extra output precision,* $\delta_{\mathrm{mid}} = 0$. *Its hypotheses hold along the root family below.*

Proof. Theorem 6.4 applies to the bridge's disk input. Since $\ell \leq k$, its error has the stated bound, with unchanged order and width. Lemma 7.1 checks all size conditions, including lifts. □

7 Multiplier Assembly and Proof of Theorem 1.1

We choose a *root family*, meaning parameters indexed by input length n, with volume $V = \Theta(n)$ and valid butterfly interfaces. All primitive, size, precision, and cost checks hold above a fixed *activation cutoff* n_*; smaller inputs use the independent multiplier.

For large n, put $b = \lfloor \log_2 n \rfloor$, $N_{\mathrm{dig}} = \lceil n/b \rceil$, and let $R = 2^k$ be the least power of two at least $2N_{\mathrm{dig}}$. For fixed positive rationals ε, c, set the accuracy target $p = 2b + 2k + 6$ and choose $d = \lfloor k^\varepsilon \rfloor$,

$\ell = \lceil k/d \rceil$, $r = 2^\ell$, and $K = \lfloor d^c \rfloor$. Choose the first $d\ell - k$ axes of length $r/2$ and the others of length r. Then $0 \le d\ell - k < d$, $t_d = r$, and $\prod_i t_i = R$. Take (ω, P, q, w) from (8). Compute rational floors by integer comparisons: for $\varepsilon = \varepsilon_1/\varepsilon_2$, d is the largest integer with $d^{\varepsilon_2} \le k^{\varepsilon_1}$, and similarly for K.

The next lemma checks sizes and headers at both working precisions.

Lemma 7.1 (Uniform root family)**.** *Fix $0 < \varepsilon < 1/14$, $c > 0$, and $\varepsilon(1+c) < 1$. For all sufficiently large n, $RP = \Theta(n)$, $P = \Theta(q) = \Theta(k) = \Theta(\log n)$, $d = \Theta(P^\varepsilon)$, $K = \Theta(P^{\varepsilon c})$, and $\ell = \Theta(P^{1-\varepsilon})$. At each $q' \in \{q, q+8d\}$, the size bands (5) and the dimensional and header conditions of Theorem 6.4 hold at width $q'+2$, with*

$$(a_d, b_d, a_r, b_r, C_\ell, C_M, C_w, C_{\mathrm{desc}}) = (\tfrac{1}{4}, 1, \tfrac{1}{128}, \tfrac{1}{8}, 1, 2, 2, 128).$$

Moreover, $dK = o(P)$, $K = o(\ell)$, $K/\log P \to \infty$, and $r/P^j \to \infty$ for every fixed j.

Proof. From $2N_{\mathrm{dig}} \le R < 4N_{\mathrm{dig}}$, we get $R = \Theta(n/\log n)$ and, on taking logarithms, $b/k \to 1$. Together with $2^b \le n < 2^{b+1}$, these bounds give $k \le b \le 2k$ for $k \ge 4096$. Substitution yields $P = 4b + 16k + 32\lg(2b+3k+8) + 268$. Hence $P/k \to 20$, $RP = \Theta(n)$, and $16k \le q \le 32k$; $d \le k$ then gives $16k \le q' \le 64k$. Once $k^\varepsilon \ge 2$, we have $k^\varepsilon/2 \le d \le k^\varepsilon$. Since $64^\varepsilon < 2$, this gives $(q')^\varepsilon/4 \le d \le (q')^\varepsilon$. Also $k/d \le \ell \le 2k/d$, since $k/d \ge 1$, hence $q'/(128d) \le \ell \le q'/(8d)$.

The remaining size estimates follow by taking powers and floors. Since $\varepsilon(1+c) < 1$, we have $dK = o(P)$ and $K = o(\ell)$. The two growth limits follow from $\varepsilon c > 0$ and $1 - \varepsilon > 0$. Thus eventually $d \ge 2$, $2 \le \ell < q'$, and $K \le \ell - 1$. The other dimensional bounds are $d\ell \le k + d \le 2k < q'$, $\log_2(2R/r) \le k+1 < 2q'$, and $q'+2 \le 2q'$. In each slice header, $\log_2 A + \log_2 S + \log_2 r = k - KD \le k$. Their codes and the other six fields use at most $2k + 12\lg q' + O(1) < 128q'$ bits. □

The target p allows exact coefficient recovery by integer rounding.

Proposition 7.2 (Two-call root packing)**.** *Pad the radix-2^b digit vectors $x^{(0)}, x^{(1)}$ of A, B to length R, and put $x_j = 2^{-b}(x_j^{(0)} + i x_j^{(1)})$. Compute $\hat{y} = \textsc{Bridge}(x)$, $\hat{w} = \mathcal{Q}_{p+2}(\overline{\hat{y} \odot \hat{y}})$, and $\hat{z} = \textsc{Bridge}(\hat{w})$. Then $c_j = \mathrm{round}(-2^{2b-1} R^2 \operatorname{Im} \hat{z}_j)$ is the exact ordinary digit convolution, and carry propagation returns AB. Outside the bridge calls, the cost is $O(R\mathsf{M}(P) + n)$.*

Proof. For $y = \mathcal{F}_R x$, (2) gives

$$x^{(0)} \circledast x^{(1)} = -2^{2b-1} R^2 \operatorname{Im} \mathcal{F}_R(\overline{y \odot y}),$$

since $\operatorname{Im}(x \circledast x) = 2^{1-2b}(x^{(0)} \circledast x^{(1)})$. There is no wraparound because $2N_{\mathrm{dig}} - 2 < R$.

Put $u_0 = 2^{-p}$. For simpler constants, use the weaker consequence $\|\textsc{Bridge}(v) - \mathcal{F}_R v\|_\infty < (8\|v\|_\infty + 2)u_0$ of (9). Since $p \ge \max(14, k)$, $p + 2 \ge b$, and $\|x\|_\infty < \sqrt{2}$, the first call is admissible and $\|\hat{y} - y\|_\infty < 14u_0$. Using $|\hat{y}^2 - y^2| \le |\hat{y} - y|(2|y| + |\hat{y} - y|)$ entrywise, squaring and nearest rounding give

$$\|\hat{w} - \overline{y \odot y}\|_\infty < (28\sqrt{2} + 196\,2^{-14} + \sqrt{2}/8)u_0 < 39.8u_0.$$

Thus $\|\hat{w}\|_\infty < 2 + 39.8\,2^{-14} < 2.003$. The second call is admissible and contributes less than $18.1u_0$; contraction of $\mathcal{F}_R$ gives total error below $57.9u_0$. The recovery scale is $2^{2b-1}R^2 = 2^{p-7}$, so coefficient error is less than $57.9/128 < 0.453 < 1/2$. Integer rounding is exact. For the imaginary numerator ν_j of the $(p+2)$-bit output, recovery rounds $-\nu_j/512$.

Starting with $s_0 = 0$, output $\gamma_j = (c_j + s_j) \bmod 2^b$ and set $s_{j+1} = \lfloor (c_j + s_j)/2^b \rfloor$. These equations preserve the represented integer, and $AB < 2^{bR}$ forces a zero final carry. The bound $c_j \le R(2^b - 1)^2$ gives $s_j \le R(2^b - 1)$ by induction, so coefficients and carries have $O(P)$ bits. Carries cost $O(RP) = O(n)$, and squaring uses $O(R)$ products of $O(P)$-bit integers. Packing, rescaling, and trimming are linear scans. □

Stage	Cost divided by RP	Gap
Boundary butterfly levels	dK	g_{pre}
Parallel layers and layouts	$\ell d^{\lambda'}$	g_{lay}
Combined phases and axis-d FFT	$\ell\,\mathsf{m}(P)$	g_{last}
Axis exposure and restoration	$d(1+\ell^{\tau})$	g_{axis}
Spreading and cyclic solves	$d(\mathsf{m}(P)+\mathsf{m}(P^2))$	g_{short}

Table 3: Costs of one bridge call. A gap g means $O(P^{1-g})$, up to fixed powers of $\lg P$.

7.1 Cost balance and exponent gaps

The admissibility constraints couple the leaf cutoff β, chunk width c, and layer exponent λ'. The guard width is $\Delta = \Theta(d^{C_1}) = o(P)$ when $\varepsilon C_1 < 1$, so guards and the positional lift use only $O(P)$ bits. Use a regular upper bound $\mathsf{M}(N) = AN(\lg(2N))^J$ for the fixed independent multiplier, such as [18]; constant-factor operand changes preserve it. All table and descriptor work is at most $C(R/r + r + 1)P^D = o(RP)$ for fixed C, D: $R \geq r^2/2$ and r dominates every polynomial in P.

Products in squaring, FFT phases, cyclic solves, and spreading have respective operand widths $O(P), O(P), O(P+k)$, and $O(P^2)$. For spreading, $W_t = \min(2P+1, t_a) = \Theta(P)$. Each axis has $O(R/P)$ blocks, each requiring four $O(P^2)$-bit products, so all spreads cost $O(RPd\,\mathsf{m}(P^2))$. Solves cost $O(RPd\,\mathsf{m}(P))$. Combining Lemmas 4.4 and 4.5 and Theorem 6.4 gives Table 3; pointwise squaring costs $O(RP\,\mathsf{m}(P))$ and is absorbed. No delegated product has width $\Theta(rP)$.

Substituting the root-family sizes gives

$$\begin{aligned} g_{\mathrm{pre}} &= 1-\varepsilon(1+c), & g_{\mathrm{lay}} &= \varepsilon(1-\lambda'), \quad g_{\mathrm{last}} = \varepsilon, \\ g_{\mathrm{axis}} &= (1-\varepsilon)(1-\tau), & g_{\mathrm{short}} &= 1-\varepsilon. \end{aligned} \tag{11}$$

If $g_{\min}$ is their minimum, every $\kappa < g_{\min}$ leaves positive slack for the fixed logarithmic factors.

Physical layouts would need $O(P/K)$ width-K swaps, costing $PK^{\tau-1}$ per unit volume, with gap $\varepsilon c(1-\tau)$. But (3) requires $\tau(1+c/\beta) < \lambda < 1$, hence $c < \beta(1-\tau)/\tau$. Together with $\varepsilon < 1/C_1$, this cost balance would force

$$\kappa < \varepsilon c(1-\tau) < \frac{\beta(1-\tau)^2}{C_1\tau}.$$

The deficit $1-\tau$ enters through both swap speed and chunk width. Exact positional permutations remove this cost.

For the printed constants $\beta = 1/2$, $\tau = \sigma = 1 - 2^{-50}$, $C_1 = 20$, put $u = 2^{-51}$ and choose $(\varepsilon, c, \lambda, \lambda') = (1/21, u/4, 1 - 129u/128, 1 - 63u/64)$. Here $\tau(1+c/\beta) = 1 - 3u/2 - u^2 > \tau = \sigma$ and $\sigma + \beta(1-\sigma) = 1 - u > \lambda$, so the margins in (3) and (4) are

$$\lambda - \tau(1+c/\beta) = 63u/128 + u^2 > 0, \qquad \lambda' - (1-u) = u/64 > 0.$$

Also $1 - \varepsilon C_1 = 1/21$. The other gaps, in table order, are $20/21 - u/84$, $1/21$, $40u/21$, and $20/21$. Thus $g_{\min} = g_{\mathrm{lay}} = 3u/64 = 3 \cdot 2^{-57} > 2^{-56}$, and $\kappa = 2^{-56}$ leaves slack $s = g_{\min} - \kappa = 2^{-57}$. These are the unchanged primitive constants used for the 2^{-182} bound in [16].

Remark 7.3 (Sharpened network constants certified in Lean 4)**.** The from-scratch network, leaf, and guard bounds in [11] give the following tuple for Theorem 1.2. It is not used in the written

proof of Theorem 1.1.

$$\xi = 2^{-42}, \qquad (\beta, C_1, \varepsilon, c) = (1/64, 14, 1/15, \xi/256),$$
$$(\tau, \sigma, \lambda, \lambda') = (1 - 4\xi, 1 - \xi, 1 - 63\xi/64, 1 - 31\xi/32),$$
$$\tau(1 + c/\beta) = 1 - 15\xi/4 - \xi^2 < \sigma,$$
$$\lambda - \max\{\tau, \sigma, \tau(1 + c/\beta)\} = \xi/64 > 0,$$
$$\sigma + \beta(1 - \sigma) = \lambda, \quad \lambda' - \lambda = \xi/64, \quad 1 - \varepsilon C_1 = 1/15,$$
$$(g_{\mathrm{pre}}, g_{\mathrm{last}}, g_{\mathrm{short}}) = (14/15 - \xi/3840, 1/15, 14/15),$$
$$g_{\mathrm{axis}} = 56\xi/15, \qquad g_{\min} = g_{\mathrm{lay}} = 31\xi/480,$$
$$\kappa = 2^{-46} = 30\xi/480, \qquad s = g_{\min} - \kappa = \xi/480 > 0.$$

7.2 Activation cutoff and proof

Use the printed tuple and $s = 2^{-57}$. Choose integers $C \geq 2$, $j, D \geq 1$ so that non-setup costs of two bridge calls and packing are at most $CRP\,P^{1-g_{\min}}(\lg(2P))^j$, and setup is at most $C(R/r + r + 1)P^D$. Include all polynomial per-record overhead in C, D. Let $k_{\mathrm{tape}} \geq 4096$ cover the primitive cutoffs and the size/header checks of Lemma 7.1 at both precisions, and take $C_0 \geq 1$ bounding the guard constant. Define

$$a_* = \min\{\varepsilon, \varepsilon c, 1 - \varepsilon, 1 - \varepsilon(1 + c), 1 - \varepsilon C_1, s/(2j)\},$$
$$B_* = \lceil \max\{16, 4C_0, 2C, D + 2 + \lg(8C)\} \rceil,$$
$$K_* = \max\{k_{\mathrm{tape}}, 2^{\lceil (64B_*/a_*)^2 \rceil}\}, \qquad n_* = 2^{4K_*}.$$

All entries in the minimum are positive: a_* records the smallest growth margin, and B_* bounds the overheads. For $n \geq n_*$, we have $b \geq 4K_*$ and $R \geq 2^{b+1}/b > 2^{b/2}$, hence $k \geq K_*$.

Write $z = \log_2 k \geq (64B_*/a_*)^2$. Then $a_*(\ln 2)z \geq 32B_*\sqrt{z} \geq \ln(B_*(z+6)^2)$, so $k^{a_*} \geq B_*(z+6)^2$. The first inequality uses $\ln 2 > 1/2$; the second uses $\ln B_* \leq B_*$ and $\ln(z + 6) \leq 2\sqrt{z}$ for $z \geq 1$. Using $d \geq k^{\varepsilon}/2$, $c < 1$, and $K \geq 2^{-c}k^{\varepsilon c} - 1$ gives $d \geq 2$, $K \geq 6$, and $dK \leq k^{\varepsilon(1+c)} \leq k/4$, hence $K < \ell$. The same estimate with $1 - \varepsilon C_1$ and $B_* \geq 4C_0$ gives $\Delta \leq k/4 + 1 \leq k/2$.

Since $\ell \geq k^{1-\varepsilon}$ and $\lg P \leq z + 6$, the choice $B_* \geq D + 2 + \lg(8C)$ gives $r \geq 8CP^{D+1}$. As $R \geq r^2/2$, setup is at most $5CRP^D/r < RP$, also absorbing the polynomial per-record work. Finally, $a_* \leq s/(2j)$ gives $\lg(2P) \leq z + 7 \leq k^{s/(2j)}$ and $2C \leq k^{s/2}$, so $C(\lg(2P))^j \leq k^s/2 \leq P^s/2$. All thresholds are fixed and can be chosen effectively from the primitive routines.

Theorem 1.1 (Single-grid integer multiplication)**.** *There is a deterministic multitape Turing machine with a fixed finite alphabet and a fixed finite number of tapes that multiplies two n-bit integers in worst-case time*

$$O(n(\log n)^{1-\kappa}), \qquad \kappa = 2^{-56}.$$

The machine in Algorithm 1 uses one grid $R = t_1 \cdots t_d = 2^k$, fractional-shift kernels with two-term cyclic solvers, and a direct complex tensor FFT. Above a fixed cutoff n_*, all delegated integer products have $O((\log n)^2)$-bit operands. These products and inputs below n_* use any fixed independent deterministic multiplier of cost $\mathsf{M}(N) = N(\log N)^{O(1)}$, such as [18]; there is no self-recursion.

Proof. Fix the printed tuple, cutoff, and independent multiplier M, using M directly below n_*. For $n \geq n_*$, Facts 3.5 and 3.6 apply, and Lemma 7.1 checks dimensions, widths, and headers at $q = P + 2$ and $q_* = q + 8d$.

Within each bridge, Theorem 5.6 performs $\mathcal{L}, \mathcal{U}$ exactly, and Theorem 6.4 and Corollary 6.5 supply the disk-valued middle transform with $E_{\mathrm{mid}} = 12\ell$ and unchanged width. With the axis

exposures of Lemma 4.4, Theorem 4.6 applies. Then Proposition 7.2 proves exact recovery and carry propagation in Algorithm 1. The cost table and cutoff give

$$O\Big(RP\,P^{1-g_{\min}}(\lg(2P))^j + RP\Big) = O(RP\,P^{1-\kappa}) = O\Big(n(\log n)^{1-2^{-56}}\Big),$$

using the slack $s = g_{\min} - \kappa$ and the size bounds of Lemma 7.1.

The only full grid has $R = \prod_i t_i$ points and power-of-two dimensions; the outer construction uses no CRT. Large-input delegated products have $O(P^2)$-bit operands and call the independent M, with no self-recursion. Integer comparisons determine the parameters, and Lemma A.1 gives deterministic rational tables. The imported primitives and streams use fixed finite alphabets and a fixed number of reusable tapes, so their composition is one uniform deterministic multitape machine. Signs and leading zeros cost $O(n)$; finitely many smaller lengths are absorbed into the leading constant. □

Remark 7.4 (CRT-free inner multiplication)**.** Choosing the independent two-dimensional $O(N \log N)$ multiplier of [13] makes the short inner products power-of-two and CRT-free as well. The archive [11] includes this stack through `IntMulTM.intMul_TM2_nlogn`; see Section 8. Neither the root construction nor its exponent changes.

8 Formal Verification in Lean 4 and Discussion

The Lean 4 development [4, 12, 11] proves Theorem 1.2 from scratch, including both $h = 100$ networks with rank and endpoint (input–output) certificates and all tape primitives at the constants of Remark 7.3. It also verifies the bridge, exact positional permutations, direct tensor FFT, and an independent two-dimensional $O(N \log N)$ inner multiplier [13]. No result from [16, 8] is assumed.

8.1 Target Specification and Trust Boundary

The theorem `SubNLogN.Assembly.intMul_challenge` proves `SubNLogN.Challenge.IntMul SubNLogN`. The following declarations come from the 80-line target file `SubNLogN/Challenge.lean`, in namespace `SubNLogN.Challenge` with `Computability` open. Docstrings, the decoder, and two sanity lemmas are omitted.

```
inductive PairSym
  | bit (b : Bool)
  | sep
  deriving DecidableEq, Fintype

def encodeNatPair (p : ℕ × ℕ) : List PairSym :=
  (encodeNat p.1).map PairSym.bit ++ PairSym.sep :: (encodeNat p.2).map PairSym.bit

def encNatPair : Encoding (ℕ × ℕ) PairSym where
  encode := encodeNatPair
  decode := decodeNatPair
  decode_encode := decodeNatPair_encodeNatPair

def IntMulSubNLogN : Prop :=
  ∃ M : Turing.TM2ComputableInTime encNatPair.encode encodingNatBool.encode
      (fun p : ℕ × ℕ => p.1 * p.2),
    (∀ k, Finite (M.tm.Γ k)) ∧
    ∃ C N₀ : ℕ, 0 < C ∧ ∀ L ≥ N₀,
      (M.time L) ^ (2 ^ 46) ≤ (C * L) ^ (2 ^ 46) * (Nat.log2 L) ^ (2 ^ 46 - 1)
```

The input is bits(a) `sep` bits(b), and the output is bits(ab), with all other stacks empty at the respective boundaries. Bits are least significant first, without high-order zeros; zero has the empty encoding.

The time bound $T(L) =$ `M.time L` applies to every encoded input of length L. The power 2^{46} clears the denominator of $1 - 2^{-46}$, so the target uses only $\mathbb{N}$ arithmetic. For $L \geq \max(N_0, 2)$, `Nat.log2 L` is $\lfloor \log_2 L \rfloor$, so taking 2^{46}-th roots gives

$$T(L) \leq CL\lfloor \log_2 L \rfloor^{\,1-2^{-46}} \leq CL(\log_2 L)^{\,1-2^{-46}}.$$

Mathlib's `Turing.FinTM2` supplies finite control and finitely many stacks, each accessed only at its top. The explicit `Finite` clause makes their alphabets finite, excluding unit-cost unbounded integers. For this fixed machine, symbols have constant-length binary codes, and one tape per stack simulates each transition in $O(1)$ time. Conversion to and from padded, signed n-bit representations costs $O(n)$, and $L \leq 2n + 1$ for two n-bit magnitudes. Absorbing finitely many smaller inputs gives Theorem 1.2 in the multitape model.

Proof checking trusts Lean `4.33.1`'s kernel and the standard axioms `propext`, `Classical.choice`, and `Quot.sound`. A human audit must inspect the target and seven library definitions in the pinned Mathlib revision `0df444a3`: `Turing.FinTM2`, `Turing.TM2ComputableInTime`, `Computability.encodeNat`, `Computability.decodeNat`, `encodingNatBool`, `Encoding`, and `Finite`, together with Lean core's `Nat.log2`. Proof-generating tools need not be trusted: the kernel checks their proof terms.

The archived checks [11] cover approximately 484,000 lines. All 2,011 modules passed `lake build` and fresh-kernel replay by `leanchecker` (`lean4checker`). Top-level `#print axioms` reported only these three axioms. Project scans found no uses of `sorry`, `axiom`, `native_decide`, `implemented_by`, `extern`, or `unsafe`.

Independently, `comparator` checked the exported proof against a target stored outside the source tree using both Lean and the Rust kernel `nanoda`. It rejected all seven negative-control tests: three weakened targets and four proof bypasses. These checks establish the formal statement; matching it to the intended multiplication problem remains a human responsibility.

8.2 Discussion and Open Problems

Theorem 5.6 replaces physical chunk-swap passes by exact positional permutations. For our parameter tuples, the limiting gap $\varepsilon(1 - \lambda')$ is linear in the network deficit rather than quadratic in $1-\tau$ (Section 7.1). The guard and butterfly-recursion constraints still give $\kappa < \varepsilon(1-\lambda') < (1-\sigma)/C_1$.

The $h = 100$, $m = 10^6$ networks activate only at astronomical input sizes; we claim no practical speedup. Smaller networks and guards could improve the exponent and cutoff. Further problems are to close the gap to the $\Omega(n)$ input-reading bound and to extend our positional-permutation method to transposition of single-scalar records [9]. Its present analysis relies on long payloads and cancellation of intermediate linear combinations.

8.3 AI Disclosure

Language models assisted with algorithmic simplifications, Lean proof scripts, and exposition. The authors checked the mathematical statements and the target specification and take responsibility for the work. Lean checked the formal development, and `nanoda` checked the exported proof. We do not claim a line-by-line human review of all formal sources. The full paper [5] is formally verified in Lean 4; the formal development is available in the github repository.

A Certified Power-Series Evaluation of Kernel and Phase Tables

We certify the dyadic table entries needed in Lemmas 6.1 and 4.5. Taylor remainders give rational intervals containing their true values.

Lemma A.1 (Certified evaluation of bridge and phase tables)**.** *Use the power-of-two bridge notation of Definition 4.1 and Lemma 4.3, with $R = 2^k$ and integers $P \geq 20$, $8 \leq \omega \leq P$. Put $q = P + 2$, $v = 2^{-q}$, $\mu = c_{\max}^{-1}$, and $\rho = \min(c_0, c_1)/c_{\max}$. For every bridge axis $a \geq 2$, $\delta \in A_a^{-1}\mathbb{Z} \cap [0, 1)$, $e \in \mathbb{Z}$ with $|e| \leq P$, and $u \in [t_a]$, the quantities $g_a(e + \delta), c_0, c_1, \mu, \rho$, and $e^{\pi i \vartheta}$ have rational approximations with certified absolute error at most 2^{-b} in $O((b + k)^3)$ tape steps for every integer $b \geq P$. Here the weights are for bin u, and ϑ has $O(k + \log P)$-bit numerator and denominator. At precision q:*

(i) *Folded taps in $\widetilde{\mathbb{C}}_q$ have error $< 0.71v$ relative to their sums restricted to $|s| \leq P$, and $< 0.95v < v$ relative to the full taps. Each costs $O(P(P + k)^3)$ steps.*

(ii) *Real $\hat{\mu}, \hat{\rho} \in 2^{-q}\mathbb{Z}$ have errors $< v$, with $\hat{\rho} \geq 0$.*

(iii) *Phases in $\mathbb{D}_q$ have error $< 2v$ and are exact when $2\vartheta \in \mathbb{Z}$.*

For phases alone, b-bit evaluation costs $O((b + h)^3)$ for $b \geq 8$ and h-bit rational angles; (iii) *holds for every $q \geq 8$ independently of P, ω. Thus each $2r$-th root costs $O(q^3)$ when $\log_2 r = O(q)$.*

Proof. Set $B = b + k + \lg(P + 2) + 32$, $N = S_* = 256B$, and $\eta = 2^{-S_*}$, using N terms at S_* fractional bits. For constants and arguments, round lower interval endpoints down and upper endpoints up to mesh η. Evaluate Taylor sums at dyadic midpoints, adding coordinate error bounds for complex values before final grid rounding.

Machin's identity and the logarithm series give

$$\pi = 16 \arctan(1/5) - 4 \arctan(1/239), \qquad \ln 2 = 2 \sum_{j \geq 0} \frac{3^{-2j-1}}{2j + 1}.$$

Using $\arctan x = \sum_{j \geq 0} (-1)^j x^{2j+1}/(2j + 1)$, the N-term tails are below $20 \cdot 5^{-2N}$ and 3^{-2N}, both below η. Generating powers by division gives contraction factors at most $1/9$, so outward rounding gives enclosure widths at most $2^7 N\eta$. These series also certify $3.14 < \pi < 4$ and $0.693 < \ln 2 < 0.7$. Integer square root after scaling by 2^{2S_*} encloses $\sqrt{\pi}$ and $\alpha_\pi = 2\sqrt{\omega \ln 2}$.

We reduce rational phases modulo 2 exactly, then use octant symmetries and the sine and cosine Taylor series on $[-\pi/4, \pi/4]$. For $s = e + \delta$, avoid division near zero by using $\operatorname{sinc}(s) = \sum_{j \geq 0} (-1)^j (\pi s)^{2j}/(2j + 1)!$ when $|s| \leq 1/2$. Otherwise use $(-1)^e \sin(\pi\delta)/(\pi s)$, whose denominator has magnitude greater than $\pi/2$.

For $x = \pi^2 s^2/(4\omega \ln 2)$, replace the Gaussian by zero if its certified lower bound reaches $b + 32$. This costs $< 2^{-b-32}$ error since $|\operatorname{sinc}(s)| \leq 1$ and $e^{-x} \leq 2^{-x}$ for $x \geq 0$. Otherwise its upper bound is $< b + 33$, since the argument interval has width < 1 (verified below). We use

$$e^{-x} = \sum_{j \geq 0} \frac{(-x)^j}{j!}, \qquad \operatorname{erf}(z) = \frac{2}{\sqrt{\pi}} \sum_{j \geq 0} \frac{(-1)^j z^{2j+1}}{j!(2j + 1)}.$$

The three erf arguments defining c_0, c_1 satisfy $z^2 < 9\omega \ln 2 < 7P$; oddness handles negative arguments. We evaluate N terms by recurrence. The sinc multiplier is $-(\pi s)^2/((2j + 2)(2j + 3))$; for erf, use $T_0 = z$, $T_{j+1} = z^2 T_j/(j + 1)$, then use $(-1)^j T_j/(2j + 1)$.

Each prepared argument has enclosure width $\leq 2^{24} B^3 \eta < 1$: the formulas use at most 32 local operations, with sensitivities $\leq 16B^2$ to $\pi, \ln 2$ and local rounding errors. On these ranges, the absolute Taylor sums and their first derivatives are at most $2^8 B^2 e^{8B}$. Recurrence multipliers are

$\le 26B/(j+1)$, so propagation factors and tails satisfy

$$\prod_{\nu=i+1}^{j} \frac{26B}{\nu} \le \frac{(26B)^{j-i}}{(j-i)!} \le e^{26B}, \qquad \frac{(26B)^N}{N!} < \left(\frac{78B}{N}\right)^N < 3^{-N}.$$

Here $N! > (N/e)^N$ and $e < 3$.

Summing propagated argument and rounding errors bounds each elementary evaluation by $2^{48}B^8e^{26B}\eta < 2^{-192B} < 2^{-b-20}$: for $B \ge 32$, $2^{48}B^8 < 2^{12B}$ and $e^{26B} < 2^{52B}$. Past term $N = 256B$, ratios are $< 1/8$, so each tail with its prefactor is at most $4(1+26B)3^{-N} < 2^{-b-20}$. These absolute bounds allow cancellation in the alternating series. There are $O(B)$ steps on $O(B)$-bit numbers, so schoolbook multiplication, division, and digit-by-digit square root give $O(B^3) = O((b+k)^3)$ total time since $b \ge P$.

By Lemma 4.3, $c_{\max} > 0.49999999$, and c_0 is larger exactly when $u < t_a/2$. The final products and divisions amplify elementary errors by less than 16. Evaluation and tail errors, together with the Gaussian cutoff error, give $16 \cdot 2^{-b-19} + 2^{-b-32} < 2^{-b}$, proving the unrounded claims. Evaluate μ, ρ within $v/16$, apply $\mathcal{Q}_q$, and replace any negative $\hat{\rho}$ by zero. Since $\rho \ge 0$, this replacement cannot increase error, so both errors are at most $v/16 + v/2 < v$.

For a folded tap, evaluate at most $2P+1$ retained samples at $b = q + \lg(2P+2) + 12$ and sum exactly at their common dyadic scale. The sample errors total $< 2^{-12}v$; applying $\mathcal{Q}_q$ gives error $< (2^{-12} + 1/\sqrt{2})v < 0.71v$. For omitted integer indices s, $\pi/\alpha^2 > 4/(5P)$ and $|s+\delta| > P$. Successive distances increase by one, so the Gaussian bounds decrease by a factor at most $e^{-8/5}$. Summing this geometric bound gives

$$\sum_{\substack{s\in\mathbb{Z} \\ |s|>P}} |g_a(s+\delta)| \le \frac{2e^{-4P/5}}{\pi P(1-e^{-8/5})} < \frac{8}{9P}2^{-P} \le \frac{8}{45}v < 0.24v.$$

Here $e^{8/5} > 4$ follows from the first four exponential terms, and $\ln 2 < 0.7 < 4/5$ gives $e^{-4P/5} < 2^{-P}$; use also $\pi > 3$ and $P \ge 20$. Each residue-class tail is no larger, so adding it gives full-tap error $< 0.95v$. Since $b = O(P)$, the $O(P)$ sample evaluations give the stated tap cost.

Finally, test $2\vartheta \in \mathbb{Z}$ exactly and return fourth roots exactly. Otherwise evaluate e_0 within $v/32$ of $e^{\pi i\vartheta}$ and set $\hat{e} = Q_q((1-v/8)e_0)$. Since $(1-v/8)(1+v/32) < 1$, the scaled value lies in the disk and has error $< 5v/32$. Truncation preserves the disk and adds less than $\sqrt{2}v$, giving total error $< (\sqrt{2} + 5/32)v < 2v$. For phases alone, take $B = b + h + 32$ and use only the trigonometric evaluation. The same bounds hold for $b \ge 8$; choosing $b = q + 5$ proves the disk claim for every $q \ge 8$. □

References

[1] Leo I. Bluestein. A linear filtering approach to the computation of discrete Fourier transform. *IEEE Transactions on Audio and Electroacoustics*, 18(4):451–455, 1970.

[2] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Exact-arithmetic Python reference implementation accompanying "A simpler algorithm for integer multiplication below $n \log n$, with a formally verified proof". Exact-arithmetic Python 3.13 reference implementation of Algorithm 1; directory `python/` of the repository, 2026. Available at https://github.com/lockqueue/intmul.

[3] Anindya De, Piyush P. Kurur, Chandan Saha, and Ramprasad Saptharishi. Fast integer multiplication using modular arithmetic. *SIAM Journal on Computing*, 42(2):685–699, 2013.

[4] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[5] Piaoyang Cui, Yiming Liu, and David P. Woodruff. A simpler algorithm for integer multiplication below $n \log n$, with a formally verified proof. Full paper, 2026. Available at https://github.com/lockqueue/intmul.

[6] Martin Fürer. Faster integer multiplication. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing (STOC)*, pages 57–66. ACM, 2007.
[7] Martin Fürer. Faster integer multiplication. *SIAM Journal on Computing*, 39(3):979–1005, 2009.
[8] David Harvey and Joris van der Hoeven. Integer multiplication in time $O(n \log n)$. *Annals of Mathematics*, 193(2):563–617, 2021.
[9] David Harvey and Joris van der Hoeven. Integer multiplication is at least as hard as matrix transposition. In *Proceedings of the 66th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 1366–1381. IEEE Computer Society, 2025.
[10] David Harvey, Joris van der Hoeven, and Grégoire Lecerf. Even faster integer multiplication. *Journal of Complexity*, 36:1–30, 2016.
[11] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Integer multiplication below $n \log n$ on multitape Turing machines: a Lean 4 formalization. Lean 4 formalization accompanying this paper (Lean `4.33.1`, Mathlib `0df444a3`), statement file `SubNLogN/Challenge.lean`, 2026. Available at https://github.com/lockqueue/intmul.
[12] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381. ACM, 2020.
[13] Piaoyang Cui, Yiming Liu, and David P. Woodruff. Integer multiplication in $O(n \log n)$ time on a single two-dimensional grid, with a formally verified proof. Companion manuscript, 2026. Available at https://github.com/lockqueue/intmul.
[14] Henri J. Nussbaumer and Philippe Quandalle. Computation of convolutions and discrete Fourier transforms by polynomial transforms. *IBM Journal of Research and Development*, 22(2):135–144, 1978.
[15] Henri J. Nussbaumer. Fast polynomial transform algorithms for digital convolution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(2):205–215, 1980.
[16] OpenAI. Integer multiplication below $n \log n$. OpenAI Math Release preprint, dated September 23, 2026 (`preprints/Integer-multiplication-below-n-log-n-September-23-2026/paper.pdf`), September 2026. Available at https://github.com/openai/math.
[17] OpenAI. An explicit power saving for the exact discrete Fourier transform. OpenAI Math Release preprint, 2026. Cited as [22] in [16].
[18] Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, 7:281–292, 1971.